

Popularity Bias in recommender systems

(Błąd popularności w
systemach rekomendujących)

Mateusz Garncarczyk

Praca inżynierska

Promotor: dr hab. Piotr Wnuk-Lipiński

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

June 11, 2026

Abstract

Recommendation systems face many challenges, and one of the most significant is *Popularity Bias*, which involves favouring popular items at the expense of niche ones. This phenomenon leads to a distortion of recommendation results, limits the visibility of the long tail, and hinders the model’s ability to learn more subtle user preferences. In this paper, we analyse this problem in the context of graph-based recommendation systems based on the *LightGCN* architecture.

The aim of the work was to investigate whether it is possible to mitigate *Popularity Bias* by intervening in the model’s learning process itself. To this end, we proposed and tested methods belonging to the *in-processing* category, based on the one hand on modifying the *BPR* loss function, and on the other on altering the graph structure during training. The approaches investigated included, amongst others, *Smoothed BPR*, *Inverse Propensity Scoring*, the proprietary *FastLightGCN* model, *PinSage*-inspired sampling, as well as *edge dropout*, *node dropout* mechanisms and their hybrid combinations.

The results show that reducing the *Popularity Bias* is possible, but requires a trade-off between improving results for niche items and maintaining the quality of the overall recommendation. The best methods allowed us to reduce the dominance of hits and improve the visibility of long-tail items without completely compromising global metrics. The results confirm that a carefully chosen modification of the loss function and graph structure can effectively support more balanced training of these models.

Systemy rekomendujące borykają się z wieloma problemami, a jednym z najważniejszych jest błąd popularności (ang. *Popularity Bias*), polegający na faworyzowaniu przedmiotów popularnych kosztem elementów niszowych. Zjawisko to prowadzi do zniekształcenia wyników rekomendacji, ogranicza widoczność długiego ogona i utrudnia modelowi uczenie się bardziej subtelnych preferencji użytkowników. W niniejszej pracy analizujemy ten problem w kontekście grafowych systemów rekomendujących opartych na architekturze *LightGCN*.

Celem pracy było zbadanie, czy możliwe jest łagodzenie *Popularity Bias* poprzez ingerencję w sam proces uczenia modelu. W tym celu zaproponowaliśmy i przetestowaliśmy metody należące do kategorii *in-processing*, oparte z jednej strony na modyfikacji funkcji straty *BPR*, a z drugiej na zmianie struktury grafu podczas treningu. Badane podejścia obejmowały między innymi *Smoothed BPR*, *Inverse Propensity Scoring*, autorski model *FastLightGCN*, próbkowanie inspirowane *PinSage'em*, a także mechanizmy *edge dropout*, *node dropout* oraz ich hybrydowe połączenia.

Uzyskane wyniki pokazują, że ograniczanie błędu popularności jest możliwe, jednak wymaga zachowania kompromisu pomiędzy poprawą wyników dla przedmiotów niszowych a utrzymaniem jakości globalnej rekomendacji. Najlepsze metody pozwalały osłabić dominację hitów i poprawić widoczność elementów długiego ogona bez całkowitego załamania metryk globalnych. Otrzymane rezultaty potwierdzają, że odpowiednio dobrana modyfikacja funkcji straty i struktury grafu może skutecznie wspierać bardziej zrównoważone uczenie tych modeli.

Contents

1	Introduction	7
2	LightGCN	11
3	Problem Definition	13
3.1	Data Bias	13
3.2	Model Bias	14
3.3	Significance of Popularity Bias	14
4	Brief Literature Review	17
5	Proposed Approaches	19
5.1	Smoothed BPR	19
5.2	Inverse Propensity Scoring	20
5.3	FastLightGCN	21
5.3.1	Neighborhood Explosion and Limitations of Full Propagation	21
5.3.2	Monte Carlo Approximation of the Convolutional Layer . . .	21
5.3.3	Drawbacks of the Original Sampling Model	22
5.3.4	Estimator Modification in the FastLightGCN Architecture . .	22
5.4	PinSage Random Walks and Dropouts	25
6	Experiments	27
6.1	Training Data	27
6.2	Training Process	28
6.3	Metrics	29

6.3.1	Diagnostic Metrics	30
6.3.2	Evaluation Metrics	30
6.4	Experimental Results	32
6.4.1	Smoothed BPR	33
6.4.2	Inverse Propensity Scoring	37
6.4.3	FastLightGCN	38
6.4.4	PinSage Random Walk and Dropouts	49
6.4.5	Combined Approaches	58
6.5	Summary of Experiments	64
7	Summary	67
8	Appendix	69
	Bibliography	79

Chapter 1

Introduction

Recommendation systems are currently part of many websites and internet services. They constitute a subcategory of information filtering systems whose task is to support users in the decision-making process. Their main function is to predict preferences based on past behaviors and to personalize suggested products, such as events, items, services, or advertisements, for a specific recipient. From the user’s perspective, the goal of such systems is to limit the problem of information overload [10] and to facilitate finding the most relevant offers in a very large set of available options. On the other hand, from the perspective of the implementing entity, their primary task is to achieve business goals, such as increasing sales, engagement, and customer loyalty.

We distinguish many types of recommendation systems, classified by their architecture and data processing methods. The most important paradigms include content-based filtering, knowledge-based models, collaborative filtering (*CF*) models, and hybrid solutions [1]. *CF* has attracted researchers’ attention for years because it enables the detection of latent user behavior patterns and interactions between users and items. Classic models of this type were primarily based on Matrix Factorization, but the development of deep learning has led to their evolution towards Graph Neural Networks (*GNN*).

The growing popularity of graph architectures is not accidental. Data in recommendation systems has a natural relational structure: users interact with items, and sets of such interactions can be interpreted as a bipartite graph. Thanks to this, *GNN* models can utilize both direct and multi-hop collaborative relationships by propagating preference signals through the local neighborhood of nodes. In the recommendation domain, one of the most famous models of this type is *LightGCN* [4], which simplifies classical graph architectures by abandoning linear transformations and non-linear activation functions in favor of neighborhood aggregation alone. In this work, this model serves as the primary baseline for further analysis and proposed modifications.

However, the effectiveness of recommendation systems largely depends on the characteristics of the collected data. Obtaining direct user ratings (*explicit feedback*) is costly and difficult to scale, which is why implicit feedback (such as clicks, purchase history, or time spent on a page) dominates in industrial practice. Signals of this type are easier to collect, but at the same time, they are much more ambiguous, as they do not always faithfully reflect actual user preferences. An interaction may result from chance, curiosity, temporary exposure, or simply from the way content is presented by the system’s interface. As a result, training data is naturally noisy and biased [1].

This problem becomes particularly important in graph architectures. The literature highlights that while *GNNs* achieve very good performance in many applications, in real-world environments they are trained on data that is far from ideal. Their effectiveness is negatively impacted by, among other things, imbalanced data distributions, the presence of various forms of noise, privacy constraints, and difficulties in generalizing out of distribution (*OOD*) [5]. However, in recommendation systems, structural noise (*Structure Noise*), as mentioned earlier, is of particular importance because embeddings are propagated further, regardless of whether they reflect real preferences or merely remnants of the exposure and data collection process.

The phenomenon of *Structure Noise* refers to the presence of connections that are uninformative, accidental, or introduce an erroneous signal into the graph. In the context of recommendation, some user-item interactions do not result from real interest but rather from the promotion of specific content, its higher visibility, or the sheer dominance of popular items in the system’s interface. Graph models build representations based on local topology; therefore, such perturbations do not remain local but can be amplified and propagated in subsequent layers of aggregation. As a result, the model learns not only user preferences but also structural deformations present in the input data.

One of the most important consequences of this process is popularity bias (*Popularity Bias*), which refers to the systematic favoring of popular items at the expense of elements belonging to the so-called long tail (*long tail*). In practice, this means that the model is more likely to recommend items that are frequently observed and strongly embedded in the graph structure, while rarer items, although they might be relevant and valuable to the user, are gradually pushed out of recommendation lists. This problem is significant not only from the perspective of personalization quality but also from the standpoint of diversity, representativeness, and fair treatment of less popular items. Due to the central role of this phenomenon in this work, its formal definition and a broader discussion will be presented in the next chapter.

The aim of this work is to investigate to what extent it is possible to mitigate *Popularity Bias* directly during the training phase of a graph model. The adopted research perspective focuses on methods from the *in-processing* category, which intervene in the optimization process itself, rather than modifying data before training

or transforming ready-made rankings after training. In this work, we analyze two main directions of such actions. The first concerns modifying the *BPR* loss function to alter the training signal for popular and niche items. The second involves structural regularization of the graph using sampling and dropout techniques, which aim to limit the dominance of dense topology regions and weaken the propagation of structural noise.

An important element of this work is also the method used to analyze the phenomenon. The evaluation of method quality is not limited solely to classical ranking metrics calculated globally. We also introduce a division of items into popularity groups and apply diagnostic metrics that describe embedding dynamics during training. This approach allows us to observe *Popularity Bias* not only as an effect visible in final recommendations but also as a phenomenon manifesting in the latent space of the representation learning process itself. This enables a deeper analysis of the impact of the studied methods on the behavior of graph models.

Chapter 2

LightGCN

LightGCN [4] is a *CF* model based on simplified Graph Convolutional Networks (*GCN*). In contrast to classical convolutional networks, which process the local neighborhood of pixels on a regular image grid, *GCN* models operate in non-Euclidean space, aggregating information directly from connected nodes in a graph. Crucially for recommendation systems, this model deliberately abandons feature transformations and non-linear activation functions, leaving only a mechanism for linear propagation of embeddings along the edges of the graph.

This model is based on operations on a bipartite graph $G = (V, E)$, where $V = U \cup I$. Here, V is the set of nodes containing all users U and items I , and E is the set of edges representing interactions between them.

Each relationship is represented in the form of an interaction matrix R of size M (the number of users) by N (the number of items), where $R_{ui} = 1$ indicates that user u interacted with item i , and 0 otherwise. Using the matrix constructed in this way, we create the adjacency matrix A actually used in matrix operations:

$$A = \begin{pmatrix} 0 & R \\ R^T & 0 \end{pmatrix} \quad (2.1)$$

The goal of collaborative filtering models is to learn optimal user embeddings e_u and item embeddings e_i that capture the relationship and similarity between them, which is why in each layer they draw information from their neighbors. It is worth noting that the embedding vectors $e_u^{(0)}$ and $e_i^{(0)}$ from the zeroth layer are the only model parameters that change during training. Their propagation proceeds as follows:

$$e_u^{(l+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}} e_i^{(l)}, \quad e_i^{(l+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}} e_u^{(l)} \quad (2.2)$$

where $\mathcal{N}$ denotes the node neighborhood, and $e^{(l)}$ is the node representation from layer l . The symmetric normalization factor $\frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}}$ is used to prevent the un-

controlled scale of embeddings as the number of layers (i.e., convolution operations) increases.

The final vector representations are obtained by combining embeddings from all layers:

$$e_u = \sum_{l=0}^L \alpha_l e_u^{(l)}, \quad e_i = \sum_{l=0}^L \alpha_l e_i^{(l)} \quad (2.3)$$

All of these operations are written in matrix form to speed up calculations. Let $E^{(l)}$ be of size $(M + N) \times D$ (D is the embedding length), and let P be a diagonal node degree matrix of size $(M + N) \times (M + N)$. For the operations, we also need a normalized adjacency matrix, which we define as:

$$\tilde{A} = P^{-0.5} \cdot A \cdot P^{-0.5} \quad (2.4)$$

Using the updated notation, propagation proceeds via these operations:

$$E^{(l+1)} = \tilde{A} E^{(l)} = \tilde{A}^{l+1} E^{(0)} \quad (2.5)$$

The final matrix formula for computing user and item vector representations is:

$$\begin{aligned} E &= \alpha_0 E^{(0)} + \alpha_1 E^{(1)} + \dots + \alpha_l E^{(l)} = \\ &= \alpha_0 E^{(0)} + \alpha_1 \tilde{A}^1 E^{(0)} + \dots + \alpha_l \tilde{A}^l E^{(0)} \end{aligned} \quad (2.6)$$

The authors of *LightGCN* set each α to the same value, namely $\frac{1}{L+1}$, to avoid complexity and maintain model simplicity.

The predicted interaction scores between users and items are calculated as the dot product of their vectors:

$$\hat{y}_{ui} = e_u^T e_i, \quad (2.7)$$

which are used to generate the K best recommendations for each user.

The model optimizes the embeddings using a loss function called *Bayesian Personalized Ranking (BPR)*. It utilizes prediction pairs: positive $\hat{y}_{ui}$ (the user interacted with the item) and negative $\hat{y}_{uj}$ (no interaction occurred). Its objective is to maximize the difference between them, i.e., to obtain the highest possible value for the positive item and the lowest for the negative item:

$$L_{BPR} = - \sum_{u \in U} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|E^{(0)}\|^2, \quad (2.8)$$

where λ controls the strength of L_2 regularization, and σ is the sigmoid function $\left(\sigma(x) = \frac{1}{1+e^{-x}}\right)$.

Chapter 3

Problem Definition

One of the key problems of modern recommendation systems is popularity bias, commonly referred to as *Popularity Bias*. This phenomenon consists in the disproportionate promotion of items with the highest popularity (bestsellers, hits) while ignoring less-known ones, even if those niche options would objectively be a better and more relevant choice for a given user.

The sources of this phenomenon can be analyzed from many perspectives, but in this work, we focus primarily on two of them: *data bias* and *model bias* [7]. This choice stems directly from the nature of the proposed mitigation methods, which intervene, on the one hand, in the structure of the interaction graph, and on the other, in the training function and the optimization signal generation process.

3.1 Data Bias

Data bias is the distortion present at the level of training data. Algorithms are very rarely trained on complete information about actual user preferences. In practice, they mainly use the previously mentioned (in Chapter 1) implicit feedback data, which is noisy, and its absence does not automatically indicate a negative preference, but often merely a lack of exposure to that item.

In this sense, the training data is biased by the historical exposure policy of the system. Items that are more visible, promoted, or placed high in rankings collect more interactions regardless of their actual relevance. In the literature, this phenomenon is described using concepts such as *exposure bias* or *position bias*. A common feature of these mechanisms is that the observed data does not constitute a clean record of user preferences, but rather a mixture of preferences and the conditions under which the user had the opportunity to encounter the given item.

Another manifestation of this problem is popularity imbalance. Popular items generate significantly more interactions than elements belonging to the long tail,

making them overrepresented in the training data. As a result, the model receives far more examples confirming the importance of hits than examples describing the behavior of the niche.

3.2 Model Bias

Model bias is the distortion that arises in the learning process itself. Recommendation algorithms learn from the frequency and structure of observed interactions; therefore, items that occur more frequently in the data are updated more often and more strongly than rare items. Their vector representations converge faster, are better estimated, and achieve a more stable position in the embedding space, while niche items remain poorly represented due to the limited number of observations.

From an optimization perspective, this is a "safe" solution for the model because the loss function rewards patterns most frequently present in the data, which allows global metrics to improve even when this happens at the expense of personalization quality for long-tail items.

In systems based on graph neural networks, this mechanism can be further amplified by the graph topology: popular items typically have a higher degree, participate more frequently in neighborhood propagation, and thus exert a stronger influence on the final embedding representations. Consequently, the model not only reflects the existing data imbalance but also reinforces and amplifies it through the learning mechanism itself.

3.3 Significance of Popularity Bias

Mitigating *Popularity Bias* is important from the perspective of both the user and the recommendation provider. For the user, excessive promotion of hits leads to a drop in novelty, limits personalization, and reduces the chance of discovering items better suited to their individual, often niche preferences. The system thereby becomes more predictable and less useful because, instead of assisting the discovery process, it repeatedly exposes the same narrow group of popular items.

From the perspective of the recommendation provider, this problem also has a highly practical and economic dimension. The platform typically possesses a wide product catalog, a significant portion of which belongs to the long tail—items that are selected less frequently and are less exposed. If the system consistently promotes only hits, a large part of the inventory, offers, or content remains unused, taking up space and creating risk of losses. Mitigating *Popularity Bias* thus not only improves the relevance of recommendations for the user but also allows for more effective monetization of the niche catalog and better utilization of the full spectrum of the available assortment.

In this context, the phenomenon of *rediscovered gems*—the rediscovery of valuable but previously under-exposed items—becomes particularly important. For the user, this means a higher chance of finding relevant and less obvious recommendations, while for the platform, it presents an opportunity to activate idle assets and extract value from catalog items that would otherwise be ignored without appropriate recommendation support.

The aim of this work is to investigate the possibilities of mitigating popularity bias by weakening its two aforementioned sources. In particular, the work focuses on modifying the training function and changing the structure of the interaction graph during model training.

Chapter 4

Brief Literature Review

The literature describes three categories of *Popularity Bias* mitigation based on the stage at which a given method is applied. We distinguish: before training (*pre-processing*), during training (*in-processing*), and after training (*post-processing*).

Pre-processing approaches modify the input data or candidate set prior to actual training or ranking. These include, among others, reweighting samples associated with niche items, interaction resampling, and limiting the influence of highly popular items. As an illustration of such an intervention, one can point to the work of **Cremonesi et al.** [3], which analyzes a scenario where items belonging to the *short head* are removed, forcing the system to operate exclusively on items from the *long tail*.

In-processing approaches intervene directly in the learning mechanism of the model. They most commonly involve modifying the loss function, adding regularization, or changing the sampling strategy so that the model does not excessively reinforce popular items. An example is the work of **Koneru et al.** [8] for the *SASRec* model [6], where they applied the *gBCE* loss function, modified negative sampling, and a *top-k negative sampling* strategy, which limited *Popularity Bias* while improving business metrics.

Post-processing approaches operate after the model is trained, adjusting the final ranking or recommendation outputs. This group primarily includes *re-ranking* methods and various forms of popularity compensation, which have the advantage of being easy to deploy without needing to retrain the model. Such a mechanism was proposed by **Zhu et al.** [13] in their *Popularity Compensation* method, where final scores are adjusted considering both item popularity and predicted user preferences.

From the perspective of this work, *in-processing* methods are particularly significant because they directly influence the learning dynamics of the model. In graph-based models, the dominance of popular items can be amplified by information propagation in the graph, neighborhood aggregation, and the frequency of embedding updates, making the modification of the optimization process itself highly

justified.

Chapter 5

Proposed Approaches

Based on the classification of *Popularity Bias* mitigation methods described in Chapter 4, this work focuses on techniques from the in-processing category. The designed solution is based on two pillars. The first concerns modifying the item scoring mechanism, with a particular emphasis on protecting items from the long tail. The second pillar focuses on structural graph regularization (dropout), which consists in the stochastic removal of information (such as edges and nodes).

5.1 Smoothed BPR

The first of these is the modification of the negative sampling process in the *BPR* algorithm. Normally, this method samples every item (popular and rare) according to a uniform distribution (i.e., $p = \frac{1}{M}$). Rare items are much more numerous, meaning they are frequently selected as negative samples. In this situation, the model learns not to recommend them because hits easily outperform them, lowering their recommendation score. This problem is exacerbated when a positive niche item competes with another negative niche item for its position in the ranking. This phenomenon is called *Gradient Starvation*. It occurs when the model focuses on learning the easiest features (which reduce the loss function the fastest), ignoring latent features that lead to generalization, which manifests as *Popularity Bias*.

Smoothed BPR (SBPR) is a method that partially addresses this issue. It is based on changing the distribution from which negative items are sampled. The probability distribution is calculated as follows:

$$P(i) = \frac{deg_i^\alpha}{\sum_{j \in I} deg_j^\alpha}, \quad (5.1)$$

where: α is the smoothing parameter to which we raise each of the item degrees, and deg_i denotes the degree of item i .

The introduced parameter α serves to increase the frequency of sampling popular

items, causing these items to compete with other popular ones for their position, while rare items have a chance to rank relatively higher in recommendations since they are no longer penalized as heavily.

The above approach does not completely eliminate the *Gradient Starvation* problem. While the smoothing parameter handles negative sampling, hits still dominate as positives in the training data, resulting in their vectors being updated with disproportionately greater force than niche ones. In the next section, we will discuss how to balance this disparity.

5.2 Inverse Propensity Scoring

Inverse Propensity Scoring (IPS) is a statistical technique derived from studies on causal inference. In the context of recommendation systems, we analyze the impact of the training strategy on the exposure of items in recommendations and their quality. Our method involves weighting the loss function inversely proportional to the popularity of the positive item, which is referred to as the estimator of its exposure probability (*propensity score*). The weight for each item is calculated based on the following formula:

$$w_i = \min \left(\frac{\max_{j \in I} \deg_j}{\deg_i}, C \right), \quad (5.2)$$

where C is the maximum allowed weight value (clipping threshold). We apply these weights to the modified *BPR*:

$$L_{IPS} = - \sum_{u \in U} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} w_i \cdot \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|E^{(0)}\|^2, \quad (5.3)$$

where the variables λ , σ , and $\hat{y}$ are defined in Equation 2.8.

The parameter C ensures that the weight does not reach extreme values for highly niche items. Without this approach, a sudden spike in gradient variance could occur, destabilizing the optimizer and causing the learning process to collapse.

Thanks to this mechanism, the learning signal for niche positive items is amplified, allowing them to compete better with hits in the ranking. This method perfectly complements *Smoothed BPR*—while the former weakens the position of popular negative items, *IPS* focuses on adjusting the weights for positive items.

5.3 FastLightGCN

5.3.1 Neighborhood Explosion and Limitations of Full Propagation

The **FastGCN** method [2] was originally designed to address the neighborhood explosion problem (*Neighborhood Explosion*), which is the exponential growth in the number of aggregated vertices in deep convolutional layers. This represents one of the most critical performance challenges in the field of graph neural networks. By the nature of this architecture, the embedding propagation phase typically requires the entire graph to be loaded into memory. In the case of massive industrial graphs, this is often technically impossible due to system scalability requirements and the unacceptable memory and time cost of model training.

In the context of recommendation systems, full aggregation of neighbors carries a severe side-risk—it can drastically propagate and amplify popularity bias (*Popularity Bias*). This is due to the fact that hits dominate this process and exert a significantly stronger influence on the final vector representations.

5.3.2 Monte Carlo Approximation of the Convolutional Layer

To overcome these limitations, a stochastic learning model based on layer-wise sampling (*layer-wise sampling*) was introduced, eliminating the need for full aggregation of the entire neighborhood. Enabling the drawing of independent samples without breaking propagation continuity required redefining classical graph convolution. Instead of discrete sparse matrix multiplication operations, embedding propagation was interpreted as a continuous integral transform (*integral transform*) with respect to a probability measure. This paradigm shift opened the door to using **Monte Carlo** estimators—a layer becomes an approximation of an integral based on sampling a subset of nodes. In a broader architectural context, this method represents an advanced form of structural *dropout*.

To minimize the estimator variance, the authors of the original method introduced an **Importance Sampling** technique. Instead of sampling nodes uniformly, the probability of selecting a given vertex u , denoted as $q(u)$, was defined proportionally to its degree:

$$q(u) = \left\| \tilde{A}(:, u) \right\|^2 / \sum_{u' \in V} \left\| \tilde{A}(:, u') \right\|^2 \approx \frac{\deg_u^2}{\sum_{u' \in V} \deg_{u'}^2}, \quad (5.4)$$

where $\tilde{A}$ is defined in Equation 2.4. Using this distribution, the update in the hidden layer $l + 1$ is calculated based on an independently sampled subset of nodes with a fixed size t_l :

$$H^{(l+1)}(v, :) = \sigma \left(\frac{1}{t_l} \sum_{j=1}^{t_l} \frac{\tilde{A}(v, u_j^{(l)}) H^{(l)}(u_j^{(l)}, :) W^{(l)}}{q(u_j^{(l)})} \right) \quad (5.5)$$

where $H^{(l)}(u_j^{(l)}, :)$ represents the embeddings of node u_j selected in layer l .

5.3.3 Drawbacks of the Original Sampling Model

The above algorithm successfully eliminates the *Neighborhood Explosion* problem, supporting mini-batch optimization. Nevertheless, adapting this solution to the environment of sparse recommendation graphs reveals two critical drawbacks.

First, the *Importance Sampling* mechanism (defined in Equation 5.4) by definition favors hits, which are selected for the sampling pool far more frequently. As a result, the method widens the gap in the L_2 norms of embeddings between popular and niche items, directly exacerbating *Popularity Bias*.

Second, the *FastGCN* architecture was evaluated on relatively dense homogeneous graphs (e.g., citation networks *Cora*¹ and *Pubmed*¹ or social networks like *Reddit*²), where independent node sampling for individual layers did not disrupt propagation continuity. However, in the case of sparse heterogeneous graphs (e.g., of the user-item type), the probability of a physical edge occurring between a vertex randomly selected in layer $l + 1$ and a vertex from layer l is close to zero. The vast majority of elements in the adjacency matrix $\tilde{A}(v, u_j^{(l)})$ are zero, resulting in a lack of gradient flow during backpropagation, which effectively paralyzes the optimization process.

5.3.4 Estimator Modification in the FastLightGCN Architecture

To objectively investigate the impact of sampling mechanisms on mitigating *Popularity Bias*, we maintained architectural consistency with the original baseline model (*baseline*), namely *LightGCN*. In designing the optimized estimator, we discarded the training of weight transformation matrices $W^{(l)}$ and non-linear activation functions. The baseline model does not use them, and its original creators proved that in sparse graphs of recommendation systems, these components encourage overfitting (*overfitting*), whereas linear, weightless aggregation is entirely sufficient.

As part of this work, we designed a proprietary modification of the vertex selection mechanism named **FastLightGCN**. To restore stable gradient flow, the estimator utilizes conditional sampling based on the local network topology. This concept involves sampling the set of nodes for layer l exclusively from the neighbors of the nodes selected in layer $l + 1$. This was first formally proposed in the *LADIES* (Layer-Dependent Importance Sampling) architecture [14].

¹<https://github.com/tkipf/gcn>

²<https://snap.stanford.edu/graphsage/>

The probability distribution from which the set of vertices is sampled is also modified. The classical distribution directly promotes high-degree objects. To counteract this, we proposed the **Smoothed Importance Sampling (SIS)** mechanism, inspired by the solution in Section 5.1. The probability of selecting vertex u (for layer l) is conditioned on the set of nodes from the higher layer $\mathcal{V}^{(l+1)}$. We define the neighborhood of this layer as the union of all neighbors:

$$\mathcal{N}(\mathcal{V}^{(l+1)}) = \bigcup_{v \in \mathcal{V}^{(l+1)}} \mathcal{N}(v) \quad (5.6)$$

The probability of sampling node u from this pool is then smoothed by raising the degree of each of its neighbors to the power of α :

$$P_{SIS}(u|\mathcal{V}^{(l+1)}) = \frac{|\mathcal{N}(u)|^\alpha}{\sum_{v \in \mathcal{N}(\mathcal{V}^{(l+1)})} |\mathcal{N}(v)|^\alpha} \quad (5.7)$$

The parameter α allows control over the influence of a vertex's degree on its probability of being selected. This approach decompresses the structure and to some extent reduces the probabilistic dominance of hits, allowing the model to explore items from the long tail more extensively.

The reduced propagation of the *FastLightGCN* model is carried out based on dynamically constructed sparse bipartite subgraphs. In each layer, a transition matrix $A_{sub}^{(l)}$ is constructed, defining the flow of information between the aggregating nodes $\mathcal{V}^{(l+1)}$ (target) and the sampled layer of neighbors $\mathcal{V}^{(l)}$ (source). The rows represent the recipients of the collaborative signal, and the columns represent the senders. Each edge of this matrix is scaled appropriately by the **Monte Carlo** estimator. The transformation of the hidden layer is then expressed as a matrix operation:

$$\tilde{A}_{sub}^{(l)} = \frac{A_{sub}^{(l)}}{t_l} \cdot \text{diag} \left(\left[\frac{1}{P_{SIS}(u|\mathcal{V}^{(l+1)})} \right]_{u \in \mathcal{V}^{(l)}} \right) \quad (5.8)$$

where $t_l = |\mathcal{V}^{(l)}|$ is the hyperparameter specifying the size (number of nodes) of the sampled layer l .

$$H_{batch}^{(l)} = \left(\prod_{i=0}^{x-l} \tilde{A}_{sub}^{x-i} \right) H^{(0)}(\mathcal{V}^{(l)}), \quad \text{gdzie } x = L - 1 \quad (5.9)$$

where *batch* represents the batch vertices (otherwise $\mathcal{V}^{(L)}$), $H_{batch}^{(l)}$ is the matrix of aggregated embeddings of batch vertices in layer l , $H^{(0)}(\mathcal{V}^{(l)})$ represents the embeddings for nodes sampled in layer l , and L is the number of layers in the model.

The diagonal matrix compensates for the expectation bias resulting from skipping less frequently sampled vertices in the P_{SIS} distribution. The final representation matrix used for preference prediction is the average of embeddings from each stage of the layer-wise sampling:

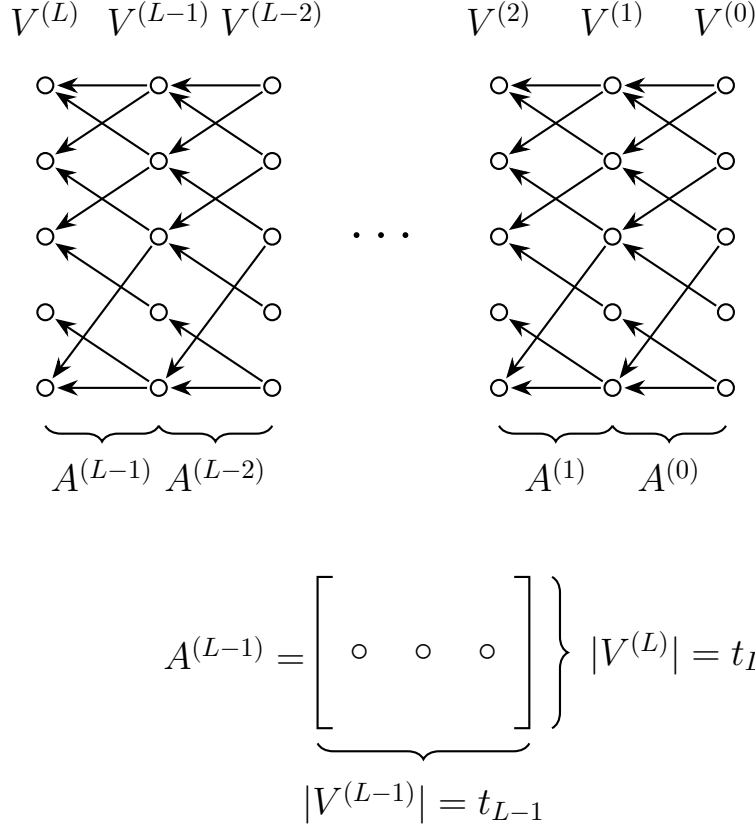


Figure 5.1: Diagram of layer-wise sampling, where propagation proceeds from the lowest layer to the highest, but node sampling is performed in reverse order. Each embedding from the sampled set of nodes for a given layer is transformed accordingly by the transition matrices $A^{(L-1)}$, whose dimensions are $t_{L-1} \times t_L$.

$$H_{final} = \frac{1}{L+1} \sum_{l=0}^L H_{batch}^{(l)} \quad (5.10)$$

Additionally, to prevent the formation of closed propagation loops (e.g., of the type $u \rightarrow i \rightarrow u$) that would artificially amplify the signal of the most popular items, we applied a sampling-without-replacement mechanism, maintaining a global set of sampled vertices $\mathcal{V}$. It enforces the uniqueness of vertices retrieved within a given batch, directly preventing redundant and repetitive selection of hits.

Taking into account the weightless nature of the baseline architecture, the distribution based on the defined *SIS* layout, the topological conditioning, and the enforced uniqueness of sampling, it can be concluded that the model is forced to build stable embeddings without relying solely on the dense neighborhood of hits.

5.4 PinSage Random Walks and Dropouts

The **PinSage** model [12] was designed for a similar purpose as *FastGCN*: both approaches aim to limit the computational cost and noise arising from training graph neural networks on massive graphs. Unlike classical graph convolutional networks, *PinSage* does not use a node’s full neighborhood, but instead constructs a local, randomly sampled subgraph. This sampling is performed using short random walk paths (*random walks*) initiated from each node belonging to the training batch. For each node, T paths (*traces*) of a fixed length H (*hops*) are generated, and the visitation frequency of individual nodes is counted. Based on this, the K most frequently visited neighbors are selected and assigned weights used in subsequent information aggregation (known as *importance pooling*).

Such an approach can be interpreted as a type of dropout at the graph structure level. Instead of the node’s full neighborhood, the model only sees a sample of the local environment, which alters the distribution of information reaching the aggregation layers, filters out accidental connections, and retains only those that are structurally significant.

Based on this, we implemented a proprietary sampling mechanism for the entire graph, differing from *PinSage* in that it is applied simultaneously to all nodes. It consists in making the K most frequently visited neighbors of each node its direct neighbors in a new adjacency matrix A . The matrix reduced in this way must then be re-normalized symmetrically, as in Equation 2.4. An additional regularization parameter is the frequency of graph reconstruction (every X epochs). This means the model does not continuously train on an identical structure, but periodically receives a new adjacency matrix, introducing desirable stochasticity and preventing overfitting to a single, rigid topology.

Intuitively, this limits the dominance of very dense graph regions by thinning their structure. Simultaneously, it increases the relative role of nodes from less populated structural fragments, granting them artificial neighbors and enriching their embeddings. As a result, the degree distribution in the training set becomes more even, which mitigates *Popularity Bias*.

Other methods of graph regularization are *Edge Dropout* and *Node Dropout*. Both mechanisms consist in temporarily removing elements of the graph structure in each epoch with a drop probability p , while scaling the retained values by a factor of $\frac{1}{1-p}$ to preserve the expected value of the signal. In the former case, individual edges are removed, while the latter removes an entire node along with all its incident edges.

While edge removal alone results in reducing the growth rate of the L_2 embedding norm for popular elements, allowing more stable learning of relationships in the long tail, node removal is more radical because it deletes all edges associated with it, forcing even stronger diversification of representations and extracting dependen-

cies for niche items. This provides a different perspective on the *Popularity Bias* problem.

Chapter 6

Experiments

6.1 Training Data

To evaluate the methods listed in the previous chapter, we use the *Gowalla* dataset¹, which was prepared by the creators of the *NGCF* architecture [11] and also adapted by, among others, the authors of the *LightGCN* model [4]. The dataset was created in accordance with rules covering data filtering and splitting. The original set was stripped of nodes (users, items) with fewer than 10 interactions (*10-core filtering*) in the global graph. For each user, 80% of their historical interactions were randomly assigned, while the remaining 20% constitute the test data. Table 6.1 shows the detailed sizes of the data.

	Dataset	Training set	Test set
Number of interactions	1,027,370	810,128	217,242
Unique items	40,981	40,981	38,546
Number of users	29,858	29,858	29,858
Matrix density	0.00084	0.00066	0.00019

Table 6.1: Information about the Gowalla datasets after preprocessing.

Table 6.2 and Figure 6.1 present information regarding popularity in the training set (based on which the decile groups discussed in Section 6.3 will be defined). The interpretation of these data directly shows the topological conditioning that is direct evidence of the *Popularity Bias* phenomenon.

The small difference between the mean and median item popularity is highly notable. The mean is strongly inflated by a small number of extremely popular nodes (so-called *hubs*), and the low median shows that the vast majority (more than half) of the items have a small contribution to the total number of connections. Such a

¹<https://snap.stanford.edu/data/loc-gowalla.html>

data distribution is referred to as a power-law distribution.

	User popularity	Item popularity
Minimum	8	1
Maximum	811	1415
Median	16	12
Mean	27.13	19.77

Table 6.2: Statistics of user and item popularity in the training set.

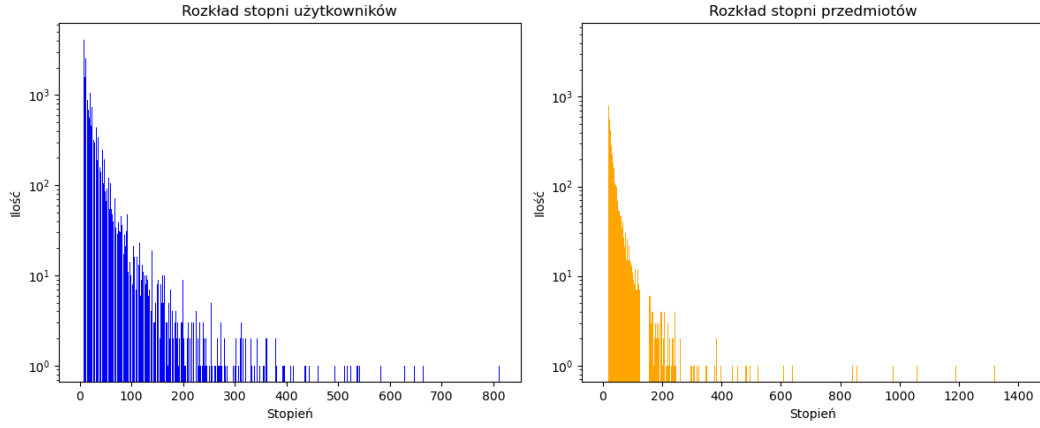


Figure 6.1: Histogram of training set popularity. The Y-axis has been scaled to logarithmic values for better readability.

6.2 Training Process

In order to maintain a fair and objective evaluation environment (*fair baseline comparison*) and isolate the influence of the studied techniques, we opted against searching the space of basic hyperparameters (*hyperparameter tuning*). Instead, for all tested solutions—both the base model, its variants with graph sampling (*FastGCN* [2], *PinSage* [12]), and modifications of the loss function (*IPS* and *SBPR*)—we adopted the optimal values experimentally found by the creators of *LightGCN*.

Table 6.3 summarizes all the shared parameters. This methodological approach ensures that the observed changes in evaluation metrics (especially within decile groups) result solely from the applied *Popularity Bias* mitigation techniques, rather than differences in model settings.

As demonstrated in the base paper, the aforementioned value of the L_2 regularization penalty is entirely sufficient to avoid *overfitting*. This allows for the complete omission of mechanisms such as node dropout (*node dropout*) and edge dropout (*edge dropout*) used, among others, in the *NGCF* architecture.

Parameter	Value
Number of layers L	4
Embedding size D	64
Embedding initialization	$\mu = 0.0, \sigma = 0.1$
Batch size	2048
Optimizer	Adam
Learning rate	10^{-3}
L_2 regularization (weight decay)	10^{-4}
Maximum number of epochs	1000
Random seed	2020

Table 6.3: Summary of basic learning process hyperparameters shared between experiments. Embeddings were initialized using a normal distribution. In *FastLightGCN*, the *batch size* is a specific parameter that affects the model structure, so its impact is discussed in Section 6.4.3.

6.3 Metrics

The main objective of this work is to analyze the impact of the individual mitigation methods described above on the Popularity Bias phenomenon. To properly evaluate recommendation quality, items were sorted in descending order based on their popularity (number of interactions) and divided into ten equal-sized deciles. Then, for evaluation purposes, these deciles were aggregated into three disjoint groups denoted as gr :

- **Top 20% (*Head*):** the set of the most popular items (so-called hits or “head”),
- **Middle 60%:** items of medium popularity,
- **Bottom 20% (*Long Tail*):** the set of the least popular items (so-called niche or long “tail”).

Applying such a division is crucial for both types of metrics used in this work. Standard global averaging of metrics makes it impossible to accurately diagnose the problem because it masks the fact that niche items are ignored in favor of popular ones.

The conducted analysis is based on two stages: **evaluation metrics** assess the accuracy of recommendations within individual decile groups, and **diagnostic metrics** examine the learning dynamics of embedding vectors during training.

6.3.1 Diagnostic Metrics

L_2 Norm

The first metric is the Euclidean norm (L_2 norm). It measures the model’s “confidence” in a given node. In most CF -type recommendation systems, such as the *LightGCN* model, the final recommendation score for a given user–item pair is calculated using the dot product of their vectors (as in formula 2.7), and then this scalar is passed to the sigmoid function σ (formula 5.1). Since this function is monotonically increasing, the longer the vector (larger values within the vector), the more easily it wins the rankings. A higher average for one group compared to another indicates that the model recommends items from the first group more strongly.

$$L_2(I_{gr}, t) = \frac{1}{|I_{gr}|} \sum_{i \in I_{gr}} \|\mathbf{e}_i^{(t)}\|_2 = \frac{1}{|I_{gr}|} \sum_{i \in I_{gr}} \sqrt{\sum_{d=0}^{D-1} (\mathbf{e}_{i,d}^{(t)})^2} \quad (6.1)$$

where I_{gr} is the set of items from the studied group, and $\mathbf{e}_{i,d}^{(t)}$ is the value of the vector of item i in training epoch t at position d .

Cosine Drift

The second tool is cosine drift (*cosine drift*). This metric measures the “mobility” (drift) of vectors, i.e., their ability to learn during training. It is calculated between the same vectors from two different epochs: the vector randomly initialized before the start of epoch zero and the vector from epoch t . If the cosine similarity of a vector during training is high relative to the initial state vector, it means that the vector did not receive enough strong gradients to learn and change its position in space relative to other nodes. This is direct evidence of the existence of the *Gradient Starvation* problem.

$$\begin{aligned} Drift(I_{gr}, t) &= \frac{1}{|I_{gr}|} \sum_{i \in I_{gr}} \frac{\mathbf{e}_i \cdot \mathbf{e}_i^{(t)}}{\|\mathbf{e}_i\|_2 \cdot \|\mathbf{e}_i^{(t)}\|_2} = \\ &= \frac{1}{|I_{gr}|} \sum_{i \in I_{gr}} \frac{\sum_{d=0}^{D-1} \mathbf{e}_{i,d} \mathbf{e}_{i,d}^{(t)}}{\sqrt{\sum_{d=0}^{D-1} \mathbf{e}_{i,d}^2} \sqrt{\sum_{d=0}^{D-1} (\mathbf{e}_{i,d}^{(t)})^2}} \end{aligned} \quad (6.2)$$

where $\mathbf{e}_i$ is the randomly initialized embedding vector of item i before the start of the learning process.

6.3.2 Evaluation Metrics

For the analysis of the effectiveness of the methods in Chapter 5, three common ranking quality metrics were used: **Recall**, **NDCG**, and **Precision**, which are cal-

culated both globally and by group. To this end, for the previously defined decile groups, the evaluation process was modified because, in reality, the model recommends items from all groups, so one cannot simply generate a ranking for only one of the groups. The modified process for generating recommendations is as follows:

1. For each user in the test set, the model selects the K top-rated items at the global level that did not appear in the training set.
2. The test interactions of user u are filtered during the creation of the relevance matrix (*relevance matrix*), where a mask of the items from decile group gr is applied, ensuring that the items belong exclusively to the tested group.
3. If, after masking, a user has no items from the studied group gr , they are not taken into account in the results.
4. The binary relevance matrix Rel_{gr} (0 for items outside group gr , and 1 otherwise) is evaluated without changes in the subsequent steps for the metrics below.

The first two of these do not take into account the order in which the items are recommended, only their relevance.

Recall

Recall measures what fraction of all existing relevant items was successfully placed in the *Top-K* recommendation list.

$$\text{Recall@K@gr} = \frac{|L_K \cap (G \cap I_{gr})|}{|G \cap I_{gr}|}, \quad (6.3)$$

where L_K is the set of the top K items for user u , and G is the set of ground truth (*ground truth*) data about their preferences. If we want the global metric, then $I_{gr} = G$.

Precision

Precision measures what fraction of the recommended items from the *Top-K* list is actually relevant to the user.

$$\text{Precision@K@gr} = \frac{|L_K \cap (G \cap I_{gr})|}{K} \quad (6.4)$$

NDCG

NDCG – *Normalized Discounted Cumulative Gain* – evaluates the quality of a given recommendation ranking, taking into account not only relevance but also the position of a given item in the list. It is defined as the ratio of the *DCG* (*Discounted Cumulative Gain*) of the ranking list to the *IDCG* (*Ideal Discounted Cumulative Gain*), which represents the ranking where items are in the ideal order.

DCG calculates the cumulative gain of a given recommendation list by introducing a penalty (logarithmic discounting) for items located at lower positions in the ranking. This ensures that the most relevant items must appear at the beginning of the list to maintain their contribution to the recommendations.

$$\text{DCG}@K@gr = \sum_{i=1}^K \frac{\text{Rel}_{gr}(L_K(i))}{\log_2(i+1)} \quad (6.5)$$

IDCG is the *DCG* with the maximum possible score for a given recommendation. It is obtained by sorting the items in descending order of their relevance (useful for rankings where we have a larger set of ratings than $\{0, 1\}$). Subsequently, this modified ranking list is substituted into the formula for *DCG*. The final formula is as follows:

$$\text{NDCG}@K@gr = \frac{\text{DCG}@K@gr}{\text{IDCG}@K@gr} \quad (6.6)$$

Thanks to normalization, the final *NDCG* score always falls within the interval $[0, 1]$, where obtaining 1 represents a perfect ranking.

6.4 Experimental Results

The goal of the experiments described in this chapter is to evaluate both custom and literature-based methods for countering the *Popularity Bias* phenomenon. To correctly interpret the gathered results, it is necessary to first define the universal objective and operational pattern of all investigated techniques (such as loss function modifications or appropriate sampling).

Their goal is not to maximize the overall accuracy of the model. The success of each of these methods is defined by finding the optimal balance point, which is a **trade-off** (*trade-off*). The desired solution is a significant increase in evaluation metrics for niche items (Bottom 20%) at the expense of reducing results for the most popular items (Top 20%). This intentional transfer of effectiveness from the “head” to the “tail” of the distribution inevitably entails a drop in global metrics. However, it is crucial that this degradation of the overall system effectiveness is controlled and

represents a rational cost for restoring the model’s ability to fairly recommend rare items.

In the following subsections, we present a detailed analysis of the impact of individual *Popularity Bias* mitigation techniques on recommendation quality and representation learning dynamics. First, we discuss the *Smoothed BPR* and *IPS* methods, followed by *FastLightGCN*, graph sampling inspired by the *PinSage* model, and the *edge dropout* and *node dropout* mechanisms, and finally, we compile the results of hybrid variants combining several techniques simultaneously.

6.4.1 Smoothed BPR

In the first series of experiments, we investigate the impact of changing the probability distribution in the *BPR* algorithm. By taking into account the node degree and the smoothing parameter during negative sampling, this method aims to temper the strength of gradients for the most popular items in the graph. We tested six different values of the parameter $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0, 1.25\}$. It is worth noting that the case of $\alpha = 0.0$ constitutes the baseline model (classic uniform sampling), with which the other variants are compared.

Table 6.4 shows that the *SBPR* method behaves as expected. The gradual increase in the value of α directly correlates with the transfer of accuracy. The optimal compromise seems to be $\alpha = 0.25$, which provides an improvement in metrics for the niche with minimal degradation of global results. Meanwhile, at the extreme value of $\alpha = 1.25$, at least a five-fold increase in metrics for the Bottom 20% group is observed compared to the baseline model. We can search for justification of the metrics improvement in the *L₂ Norm* and *Cosine Drift* plots, which investigate two different vector properties mentioned during their definition in equations 6.1 and 6.2.

The comparison of the *L₂ norm* plots (Figure 6.2) shows the impact of the parameter α on vector magnitudes. For the baseline model ($\alpha = 0.0$), the final values of vector lengths for the Bottom 20% (5.81) and Top 20% (5.73) groups are almost equalized. This confirms that niche items are unable to receive the appropriate weight updates necessary to strongly mark their presence in the vector space. This is due to their rare occurrence as positive samples in the *BPR* loss function, but frequent occurrence as negative samples. This favors losing in the ranking despite a similar vector length.

Parameter α	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
0.0 (Baseline)	0.00797	0.04942	0.33115	0.18199
0.25	0.01249	0.06008	0.31915	0.18140
0.50	0.01862	0.07084	0.29458	0.17280
0.75	0.02662	0.07933	0.26026	0.15951
1.00	0.03425	0.08803	0.22263	0.14505
1.25	0.03947	0.08999	0.19242	0.13191
NDCG@20				
0.0 (Baseline)	0.00262	0.01937	0.23210	0.15377
0.25	0.00417	0.02438	0.22735	0.15387
0.50	0.00656	0.03004	0.20927	0.14507
0.75	0.00983	0.03479	0.18181	0.13121
1.00	0.01352	0.04031	0.15259	0.11550
1.25	0.01624	0.04243	0.12772	0.10169
Precision@20				
0.0 (Baseline)	0.00067	0.00626	0.05649	0.05580
0.25	0.00107	0.00776	0.05475	0.05580
0.50	0.00159	0.00924	0.04968	0.05271
0.75	0.00229	0.01050	0.04243	0.04774
1.00	0.00301	0.01190	0.03496	0.04252
1.25	0.00342	0.01218	0.02892	0.03776

Table 6.4: The impact of the smoothing parameter α in the *Smoothed BPR* method on evaluation metrics (@20).

Increasing the smoothing parameter directly affects this process. The most noticeable phenomenon is the increasing distance between the niche and hit lines on the plots. Changing the negative sampling to a smoothed probability distribution makes popular nodes serve more frequently as reference points for penalty, which allows the rarest nodes to undergo more intensive gradient updates. This is particularly visible for $\alpha = 1.25$, where the L_2 norm for the Bottom 20% group rises to a value of 9.00 (an increase of nearly 55% compared to the baseline model). At the same time, the growth of representation for the Top 20% group slows down, reaching a value of 6.20 at the end of training. This directly proves that the *SBPR* mechanism compensates for the initial lack of interactions in the training set, serving as an effective tool for mitigating *Popularity Bias*.

The layout of the *Cosine Drift* plots (Figure 6.3) complements the conclusions drawn from the L_2 norm, illustrating the impact of the parameter α on the angular mobility of vectors during training. In the baseline model ($\alpha = 0.0$), the final cosine similarity relative to the initialization is approximately 0.044 for the Bottom 20% group and 0.061 for the Top 20% group. Low values for the niche mean that the

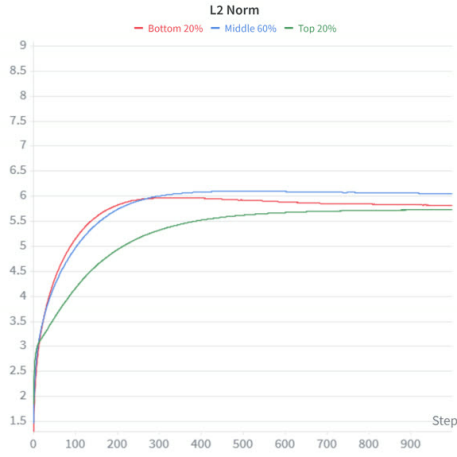
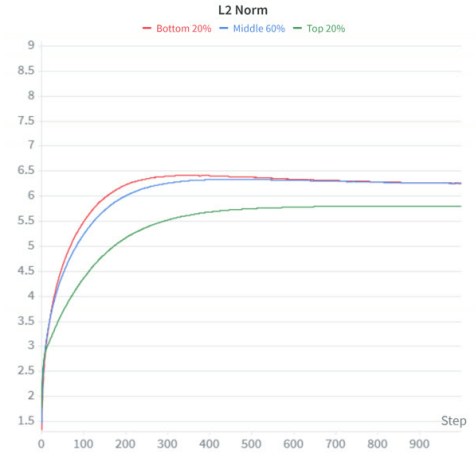
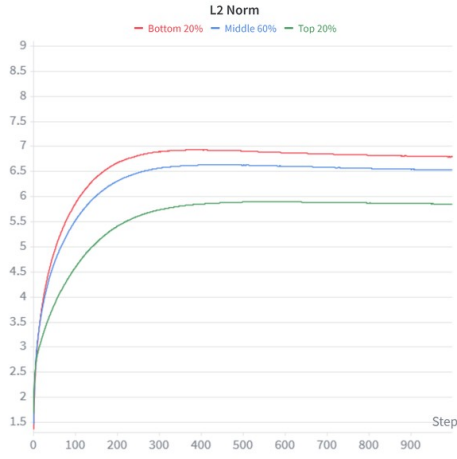
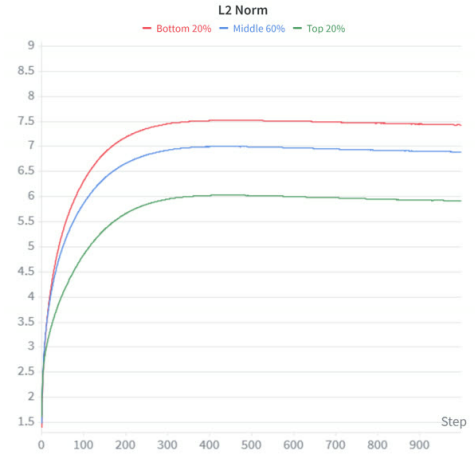
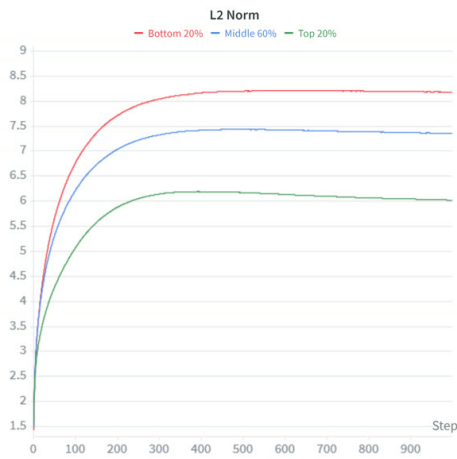
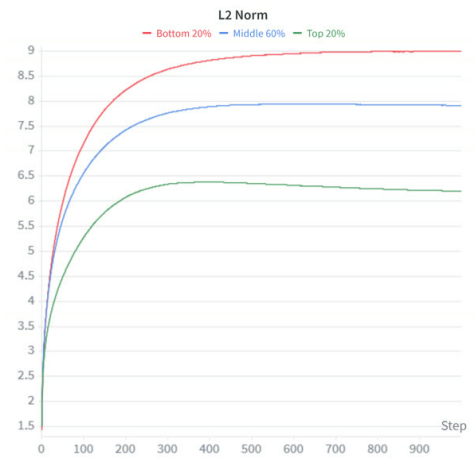
(a) $\alpha = 0.0$ (Baseline model)(b) $\alpha = 0.25$ (c) $\alpha = 0.50$ (d) $\alpha = 0.75$ (e) $\alpha = 1.00$ (f) $\alpha = 1.25$

Figure 6.2: Dynamics of the vector L_2 norm broken down by decile groups for different values of the parameter α in the *Smoothed BPR* method.

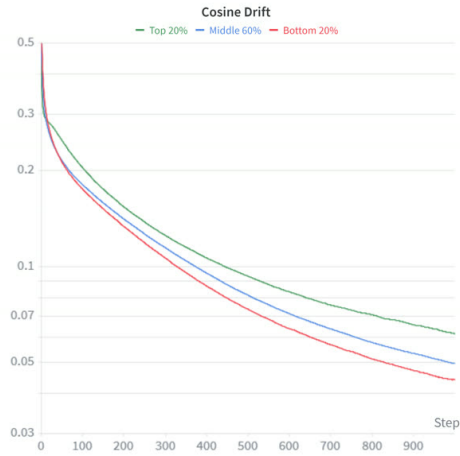
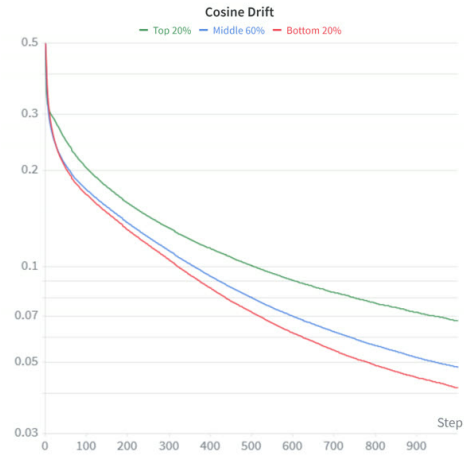
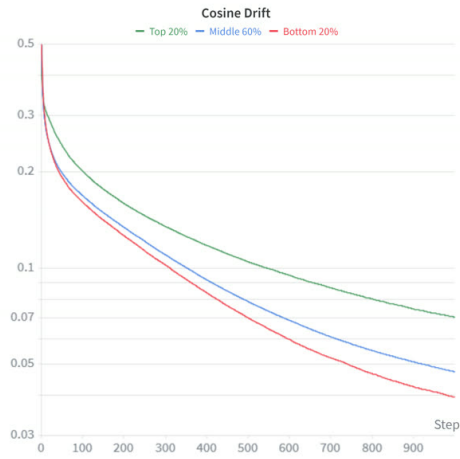
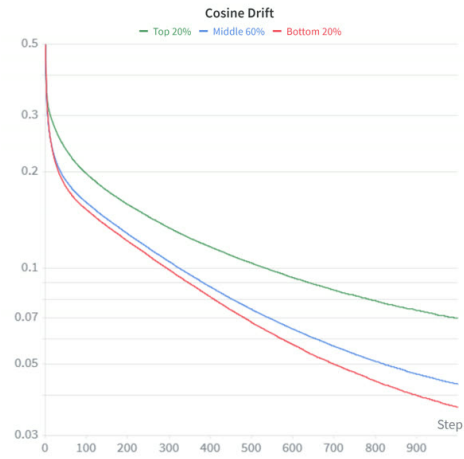
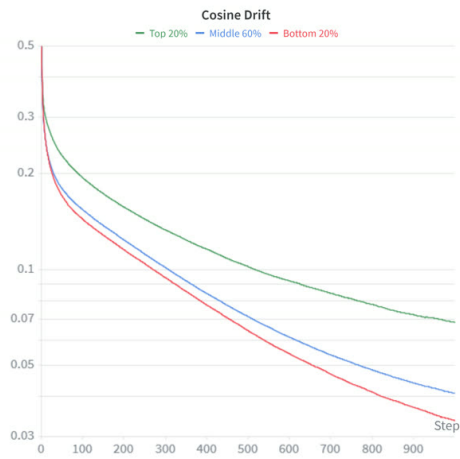
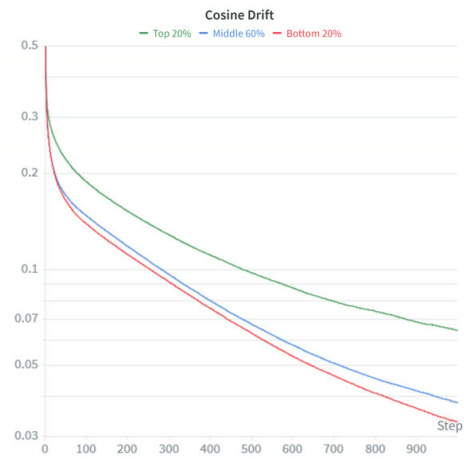
(a) $\alpha = 0.0$ (Baseline model)(b) $\alpha = 0.25$ (c) $\alpha = 0.50$ (d) $\alpha = 0.75$ (e) $\alpha = 1.00$ (f) $\alpha = 1.25$

Figure 6.3: Dynamics of the cosine drift of feature vectors broken down by decile groups for different values of the parameter α in the *Smoothed BPR* method.

vectors undergo strong rotations, which, however (as shown by the L_2 norm), lack the appropriate strength to establish the node’s position.

A systematic increase in the smoothing parameter up to the extreme value ($\alpha = 1.25$) results in a visible reduction of this indicator to 0.033 for rare items, and an increase to 0.064 for hits. Considering the specificity of this metric — where a lower value indicates a stronger change in direction relative to the vector from epoch zero — this result directly confirms the success of the applied regularization. Niche vectors receive more frequent gradient updates, which effectively reduces the *Gradient Starvation* phenomenon. At the same time, the angular shift of the hit vectors slows down, proving the correct functioning of their tempering, which ultimately allows the model to partially balance the training dynamics of the entire system.

6.4.2 Inverse Propensity Scoring

In the second series of experiments, we study the impact of weighting the loss function in the *BPR* algorithm. By assigning weights inversely proportional to the popularity of the positive item, this method aims to amplify the strength of gradients for the rarest items in the graph. We tested seven different values of the parameter limiting the estimator’s influence: $C \in \{1, 5, 10, 20, 40, 100, 200\}$. For the value of $C = 1$, the model does not significantly change its behavior; therefore, we treat this variant as the baseline model against which the remaining configurations are compared.

Analysis of the data in Table 6.5 reveals a dynamic that is largely consistent with the results presented earlier for the *SBPR* method, where the first tested value ($C = 5$) represents a trade-off, and the best of them is the last one ($C = 200$). Although the effectiveness of *IPS* is slightly reduced compared to the results of the previous method, the stark differences between the two mechanisms only reveal themselves during the analysis of the properties of the vector space itself, as confirmed by the comparison of the L_2 Norm and *Cosine Drift* measures.

The comparison of L_2 norm plots (Figure 6.4) shows the impact of weighting according to the popularity of positive items. For the variant without weighting ($C = 1.0$), the representation length for the Bottom 20% and Top 20% groups is at the same level as in Section 6.4.1. However, by consistently increasing the cutoff threshold to 200, a rapid increase in vector magnitudes across the entire graph can be observed. For this value, niche nodes build representations with a norm of around 10.94, while popular nodes soar to a level of 11.35. This is a consequence of multiplying the difference in their rankings in Equation 5.3, which forces raising the ranking of the positive item and lowering the ranking of the negative item to balance what the estimator is doing. This means that it effectively forces the model to learn that a niche item is better than a hit.

The layout of the *Cosine Drift* plots (Figure 6.5) shows a completely different direction of operation than in the *SBPR* method. Increasingly larger values of the

parameter result in a slowing down of the cosine drift from 0.044 to 0.150 (for the Bottom 20% group) and from 0.061 to 0.116 (for the Top 20% group). There is an inversion of roles in learning the angular mobility of vectors (which can be observed across the plots), where the niche learns slower and hits learn faster. L_2 Norm shows an explosion of vector lengths, while *Cosine Drift* reveals that instead of changing their location in space, the vectors stiffen and elongate in more or less the same direction.

Parameter C	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
1.0 (Baseline)	0.00791	0.04887	0.33130	0.18256
5.0	0.01648	0.06943	0.30059	0.17668
10.0	0.01976	0.07355	0.28112	0.17061
20.0	0.02432	0.07680	0.26095	0.16717
40.0	0.02655	0.07917	0.24350	0.16447
100.0	0.03193	0.08402	0.22348	0.16245
200.0	0.03842	0.08406	0.21851	0.16269
NDCG@20				
1.0 (Baseline)	0.00262	0.01941	0.23176	0.15401
5.0	0.00566	0.02933	0.21507	0.14920
10.0	0.00682	0.03214	0.20016	0.14419
20.0	0.00838	0.03454	0.18408	0.14114
40.0	0.00931	0.03693	0.16939	0.13919
100.0	0.01170	0.04086	0.15325	0.13765
200.0	0.01445	0.04128	0.14913	0.13780
Precision@20				
1.0 (Baseline)	0.00068	0.00620	0.05644	0.05592
5.0	0.00139	0.00864	0.05154	0.05376
10.0	0.00165	0.00916	0.04833	0.05161
20.0	0.00200	0.00952	0.04485	0.05013
40.0	0.00219	0.00994	0.04180	0.04914
100.0	0.00266	0.01074	0.03841	0.04848
200.0	0.00320	0.01079	0.03755	0.04864

Table 6.5: Impact of the cutoff threshold C in the *Inverse Propensity Scoring* method on evaluation metrics (@20).

6.4.3 FastLightGCN

Another architecture analyzed in this work is *FastLightGCN*, which is a proprietary adaptation of the stochastic *FastGCN* model. It utilizes layer-wise *Monte Carlo* sampling, which not only reduces the computational cost of the propagation phase, but also drastically changes the dynamics of information flow in the user-item graph.

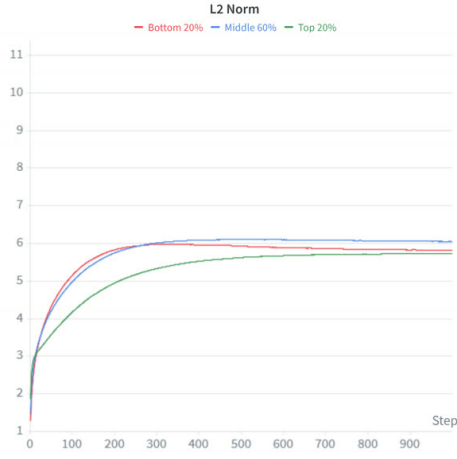
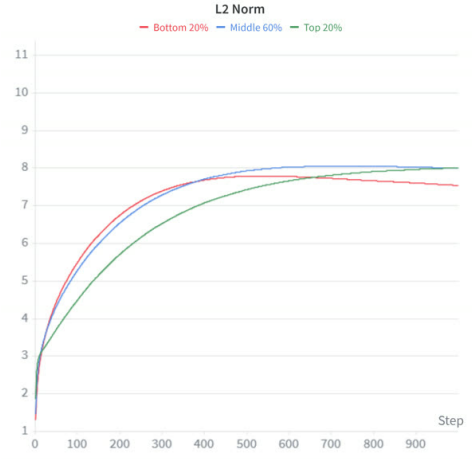
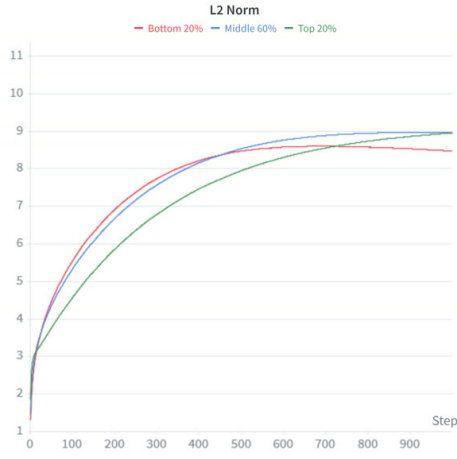
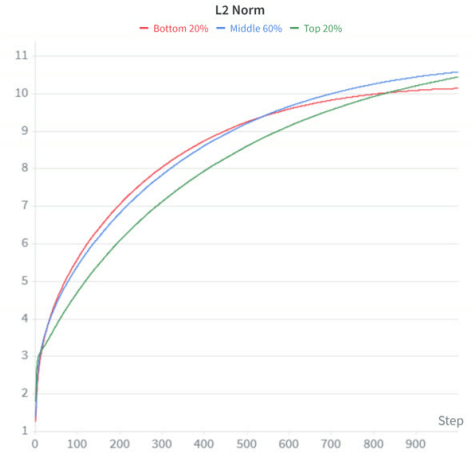
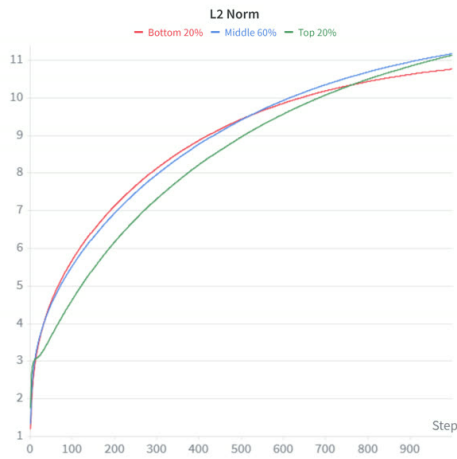
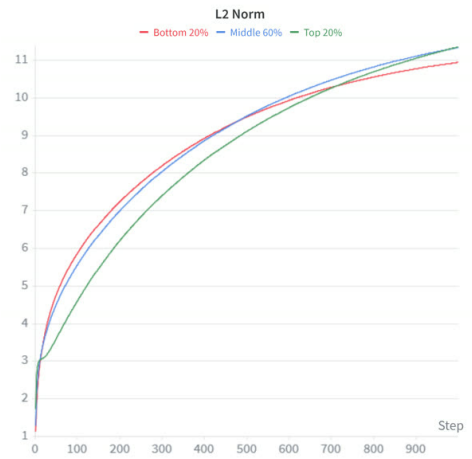
(a) $C = 1$ (Baseline model)(b) $C = 5$ (c) $C = 10$ (d) $C = 40$ (e) $C = 100$ (f) $C = 200$

Figure 6.4: Dynamics of the vector L_2 norm divided into decile groups for different values of parameter C in the *Inverse Propensity Scoring* method. The plot for $C = 20$ has been omitted as it is very similar to the plots for $C \in \{10, 40\}$.

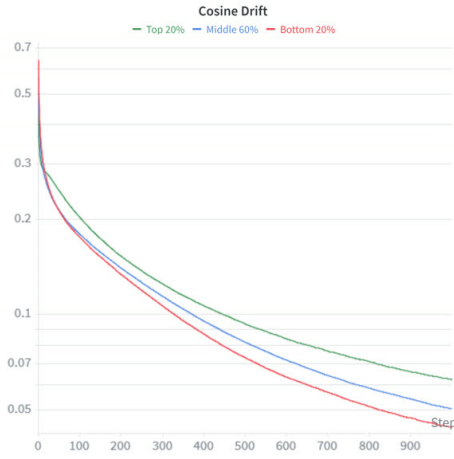
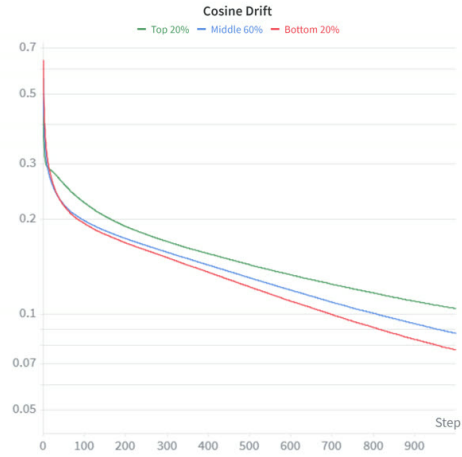
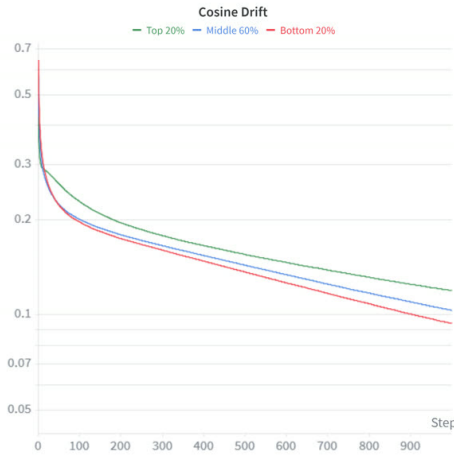
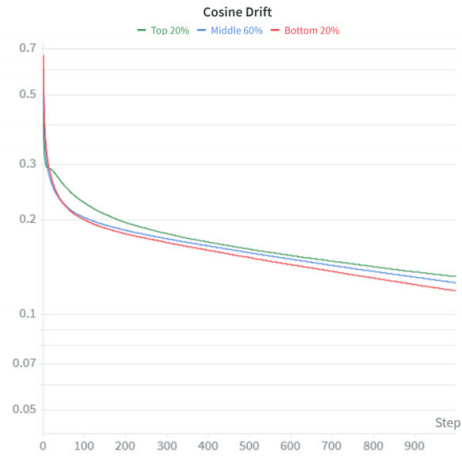
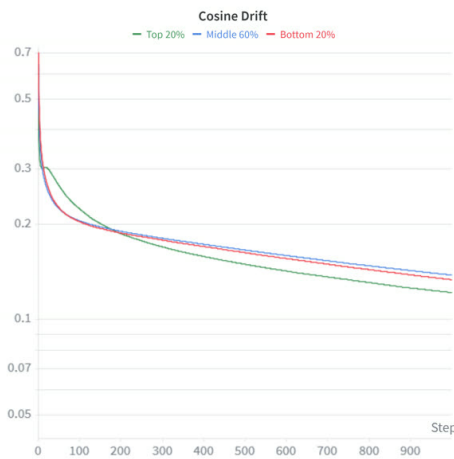
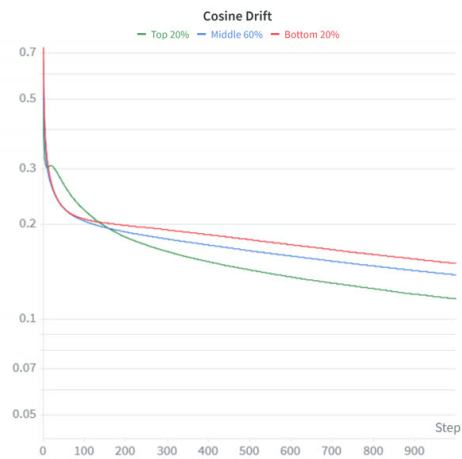
(a) $C = 1$ (Baseline model)(b) $C = 5$ (c) $C = 10$ (d) $C = 40$ (e) $C = 100$ (f) $C = 200$

Figure 6.5: Dynamics of feature vector cosine drift divided into decile groups for different values of parameter C in the *Inverse Propensity Scoring* method. The plot for $C = 20$ has been omitted as it is very similar to the plots for $C \in \{10, 40\}$.

From the perspective of this work, it is key that this type of structural *dropout* interferes with the learning process of popular nodes. By reducing the full, deterministic aggregation, the model can directly counteract the propagation of *Popularity Bias* and mitigate the impact of the *Structure Noise* phenomenon. The analysis of the model’s performance will be conducted in the following order:

1. the influence of model depth and layer width on recommendation quality within popularity groups,
2. the effect of enforcing uniqueness of sampled nodes (**unique**) on estimator stability,
3. the influence of the initialization layer width, i.e., *batch size*,
4. comparison of classical *Importance Sampling (IS)* with its smoothed version, i.e., *Smoothed Importance Sampling (SIS)*.

Model depth and width

Configuration	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
[32]	0.00273	0.01503	0.22506	0.11783
[32, 32, 32, 32]	0.00254	0.01027	0.18798	0.10115
[2048]	0.00446	0.02614	0.28896	0.15172
[2048, 2048, 2048, 2048]	0.01055	0.03810	0.28859	0.15842
NDCG@20				
[32]	0.00097	0.00651	0.14362	0.09674
[32, 32, 32, 32]	0.00101	0.00458	0.11767	0.08305
[2048]	0.00163	0.01067	0.18935	0.12345
[2048, 2048, 2048, 2048]	0.00411	0.01659	0.18998	0.12872
Precision@20				
[32]	0.00027	0.00201	0.03781	0.03610
[32, 32, 32, 32]	0.00024	0.00138	0.03306	0.03139
[2048]	0.00043	0.00330	0.04704	0.04511
[2048, 2048, 2048, 2048]	0.00095	0.00481	0.04978	0.04875

Table 6.6: Impact of propagation width and depth on evaluation metrics of the *FastLightGCN* model with *batch size* = 2048.

To implement the first point of the research plan, we investigated four variants of propagation depth (from 1 to 4) in combination with seven different sizes of the sampled layer $t_l \in \{2^5, \dots, 2^{11}\}$. It should be noted that the size of the first layer

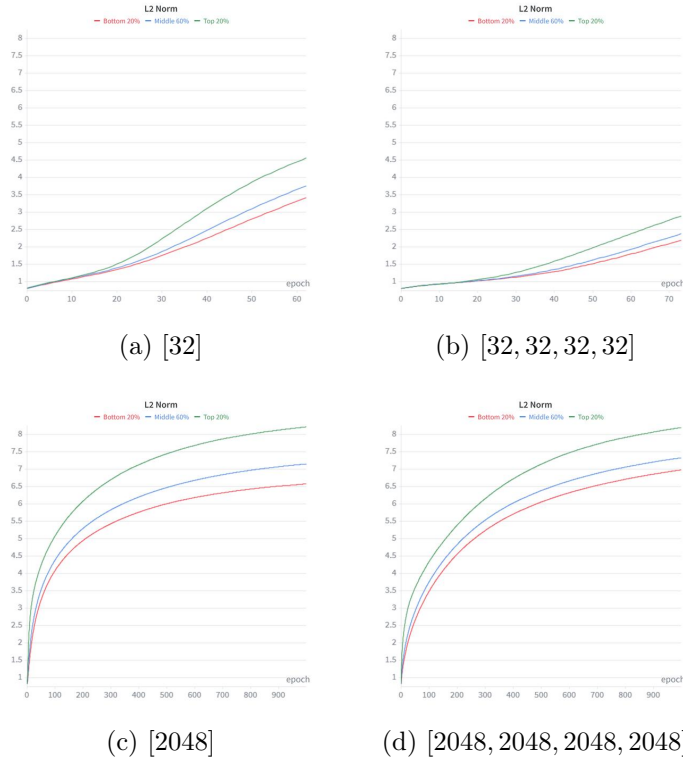


Figure 6.6: Dynamics of the L_2 norm of vectors divided into decile groups for different numbers of layers and their sizes in the *FastLightGCN* model

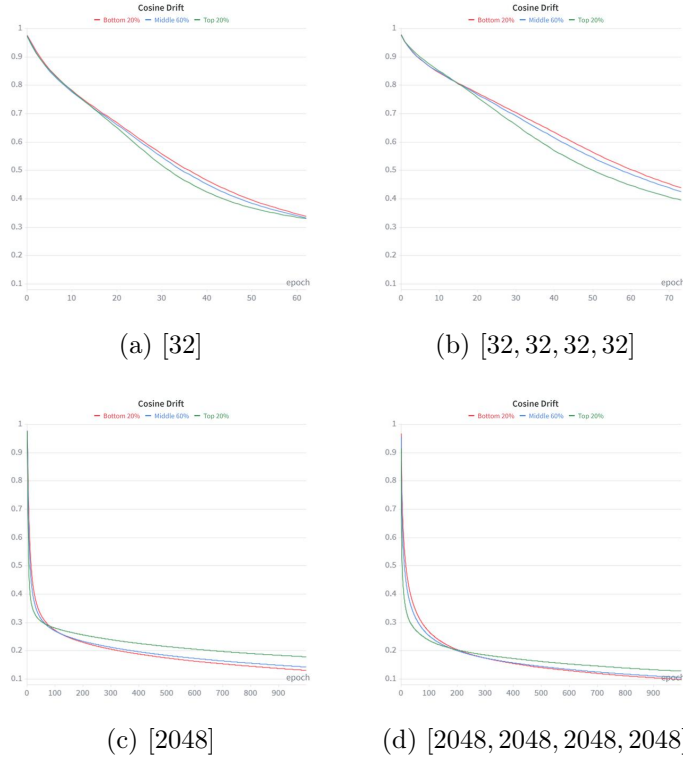


Figure 6.7: Dynamics of the cosine drift of feature vectors divided into decile groups for different numbers of layers and their sizes in the *FastLightGCN* model

in the sampled bipartite subgraphs depends strictly on the *batch size* parameter, the impact of which will be discussed later in this work. For the purposes of these experiments, we adopted the default value of this parameter from the *LightGCN* model, which is 2048.

Without loss of generality, in this analysis, we compared the extreme sampling widths (32 and 2048) for depths 1 and 4, as consistent and continuous trends are observed for intermediate values. The analysis of the presented results reveals that *FastLightGCN* alone does not match the baseline model both globally and across decile groups—this difference should be alleviated in Section 6.4.5, which is dedicated to combined mitigation methods. Regardless of this apparent performance drop, we analyze the mechanics of this model to verify to what extent stochastic propagation affects the *Popularity Bias*.

The reason for this behavior is a fundamental limitation of multi-layer node sampling in sparse graphs. The data in Table 6.6 shows that for narrow sampling variants (e.g., width 32), increasing the network depth leads to a clear degradation of recommendation quality, affecting primarily the Middle 60% and Top 20% groups, with only a negligible change in results for the Bottom 20% group. This phenomenon can be interpreted as a consequence of the accumulating approximation error of the *Monte Carlo* estimator. In Equation 5.9, each propagation layer is described by a separate transition matrix containing appropriately scaled sampling probabilities. Cascade multiplication of these matrices at small sample sizes causes rapid noise accumulation and gradual blurring of the collaborative signal. Interestingly, this effect disappears for the widest variant (2048), where the approximation is accurate enough that greater depth significantly improves the result.

The negative impact of noise propagation can be directly observed in Figure 8.2. It shows that deeper models, in which we applied narrow sampling, maintain higher and highly unstable loss function values. This indicates significant optimization difficulties and a high susceptibility of the training process to the massive variance introduced by the strict limitation on the number of sampled neighbors.

A detailed insight into this error is provided by diagnostic metrics that allow for the interpretation of vector representations. Analyzing the overall values of the L_2 Norm (Figure 6.6) and *Cosine Drift* (Figure 6.7), one can clearly see how the layer size determines the stability of representations across the embedding space. In cases where we choose wide sampling combined with greater depth ([2048, 2048, 2048, 2048]), vectors learn stably: their L_2 norm grows naturally, and the drift remains high, allowing for the formation of a coherent space. In contrast, in variants based on narrow sampling ([32, 32, 32, 32]), this phenomenon is drastically reversed. Limiting the number of sampled neighbors causes the vectors to experience weak drift, and their length is significantly reduced.

The above picture allows us to formulate a coherent conclusion: optimizing a deep architecture with a randomized mechanism only makes sense when the sam-

pling width guarantees a precise approximation of the neighborhood. Otherwise, the stochasticity of cascaded layers completely destroys the structure of the embedding space, so that for narrow sampling, a shallow model remains the optimal and safest design choice, as it naturally buffers the accumulation of error variance.

Parameter unique

Configuration	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
[2048]	0.00446	0.02614	0.28896	0.15172
[2048]-unique	0.00460	0.02543	0.28609	0.15044
[2048, 2048, 2048, 2048]	0.01055	0.03810	0.28859	0.15842
[2048, 2048, 2048, 2048]-unique	0.00929	0.02548	0.24600	0.13632
NDCG@20				
[2048]	0.00163	0.01067	0.18935	0.12345
[2048]-unique	0.00165	0.01038	0.18552	0.12099
[2048, 2048, 2048, 2048]	0.00411	0.01659	0.18998	0.12872
[2048, 2048, 2048, 2048]-unique	0.00404	0.01187	0.13477	0.09317
Precision@20				
[2048]	0.00043	0.00330	0.04704	0.04511
[2048]-unique	0.00042	0.00320	0.04618	0.04435
[2048, 2048, 2048, 2048]	0.00095	0.00481	0.04978	0.04875
[2048, 2048, 2048, 2048]-unique	0.00087	0.00312	0.04080	0.03944

Table 6.7: The impact of enforcing the uniqueness of sampled nodes on the evaluation metrics of the *FastLightGCN* model for configurations based on a layer width of 2048 and a *batch size* of 2048.

In the remainder of the analysis, we limit ourselves to the [2048, 2048, 2048, 2048] configuration to preserve the clarity of the presentation of the results, while focusing on the variant most representative of the previously observed trends.

Moving on to the analysis of the **unique** variant described in Section 5.3.4, the comparison of results highlights its close dependence on the propagation depth. For a single layer, this mechanism naturally does not affect the metrics, since neighbor sampling for a given node is already performed without replacement. The effects of deduplication only become apparent in deeper architectures.

Maintaining a global set of sampled nodes $\mathcal{V}$ for multiple layers drastically degrades the model quality, as illustrated by the collapse of the global metrics. This is due to the fact that strictly blocking the re-sampling of nodes in subsequent propagation steps deprives the subgraph of its densely connected nature. The paths of

signal flow stop intersecting naturally, and the structure of the sampled graph begins to resemble a set of disjoint paths – a kind of forest of trees whose roots are the nodes from the initial batch. Such a drastic restriction of information flow destroys the neighborhood approximation and proves that enforcing global node uniqueness in deep sampling does more harm than good.

Initialization layer (*batch size*)

Batch size	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
2048	0.01055	0.03810	0.28859	0.15842
1024	0.01227	0.04105	0.29148	0.16203
512	0.01522	0.04624	0.29439	0.16673
NDCG@20				
2048	0.00411	0.01659	0.18998	0.12872
1024	0.00482	0.01816	0.19334	0.13209
512	0.00606	0.01997	0.19489	0.13543
Precision@20				
2048	0.00095	0.00481	0.04978	0.04875
1024	0.00110	0.00519	0.05072	0.05003
512	0.00132	0.00578	0.05135	0.05136

Table 6.8: Influence of the *batch size* parameter on the evaluation metrics of the *FastLightGCN* model for the [2048, 2048, 2048, 2048] configuration.

Another investigated hyperparameter is the *batch size*, which affects not only the number of input nodes in the batch but also, indirectly, the size of the first layer in the sampled bipartite graph. This parameter is of key importance for the scale of stochastic learning: reducing the batch size means that during a single epoch, the model generates and processes more subgraphs, which leads to much more frequent updates of representations using the gradient.

The comparison of results in Table 6.8 reveals a highly beneficial impact of this mechanism on the performance of the model. In contrast to the phenomena observed in classical architectures, reducing the *batch size* value (from 2048 to 512) leads to a systematic improvement in the global quality of recommendations. Particular attention is drawn to the behavior of individual popularity groups: while the metrics for the Top 20% group improve only marginally, the results of the Bottom 20% group record a jump of nearly 50%. This suggests that smaller batch sizes, leading to more frequent updates, effectively break the dominance of the most popular items, drastically increasing the exposure of niche objects without simultaneous degradation of top-performing items.

The above conclusion is perfectly reflected in the analysis of the geometry of the embedding space, presented in Figure 6.8. As the *batch size* decreases, we observe a clear increase in the L_2 norm across all popularity groups. What is crucial for the *Popularity Bias* phenomenon, however, is that more frequent updates lead to the blurring of disparities in vector lengths. For a large batch size (2048), the length of the Top 20% vectors (8.19) significantly dominated over the Bottom 20% group (6.97). Reducing the batch size to 512 causes niche vectors to grow dynamically enough (9.27) to almost level with the top hits (9.59). This means a physical equalization of the vector influence strength in space.

At the same time, the *Cosine Drift* analysis indicates that smaller batches stabilize the training direction. Although the Top 20% group consistently exhibits the strongest angular deviation relative to initialization, the absolute value of this drift for all groups systematically decreases as the *batch size* is reduced. We interpret this phenomenon as a direct consequence of the drastic improvement in the quality of the *Monte Carlo* approximation in relation to the batch size. When the *batch size* is reduced to 512 while maintaining a constant sampling width of 2048, the first propagation layer is four times larger than the input layer. As a result, for each node in the batch, there is a proportionally much wider and more information-rich neighborhood than in the baseline variant (2048 to 2048). This increased precision of the estimator, combined with more frequent gradient updates within the epoch, effectively stabilizes the trajectory of embeddings. Reducing the batch size, therefore, turns out to be not only an action aimed at equalizing vector strength and mitigating *Popularity Bias*, but also a fundamental procedure improving the quality of stochastic propagation in the *FastLightGCN* architecture.

Smoothed SIS Probability

The final investigated component of the *FastLightGCN* architecture is the *Smoothed Importance Sampling (SIS)* mechanism. In our experiments, we analyzed the impact of distribution smoothing on the training process, taking sampling linearly proportional to the node degree ($\alpha = 1.0$) as the starting point, and then systematically reducing this parameter until a uniform distribution ($\alpha = 0.0$) was achieved.

A key conclusion from the analysis is that the use of sampling that favors high-degree nodes achieves a sort of success in targeting the *Popularity Bias* phenomenon. The comparison of the results shows that the $\alpha = 1.0$ variant drastically and selectively reduces the recommendation quality for the most popular items (the Top 20% group) and for items in the middle of the distribution (Middle 60%). At the same time, the metrics for the long-tail group (Bottom 20%) remain at a stable level. However, this aggressive suppression of popular items introduces huge variance into the estimator, which directly translates to an extremely noisy and unstable loss function curve, presented in Figure 8.2. Systematic smoothing of the distribution (decreasing the parameter α towards 0.0) effectively eliminates this gradient noise, leading to a

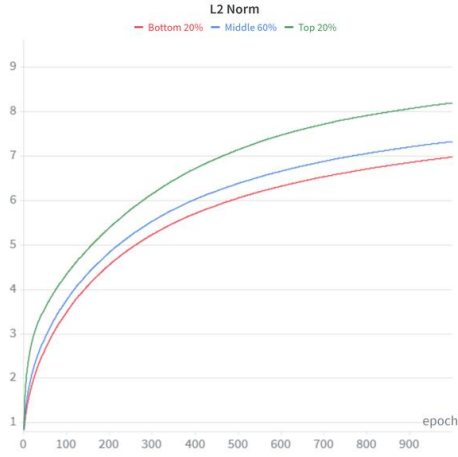
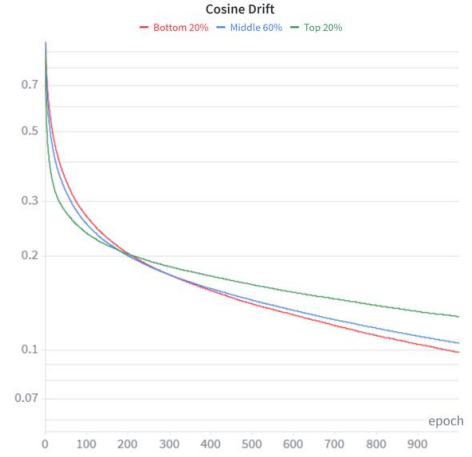
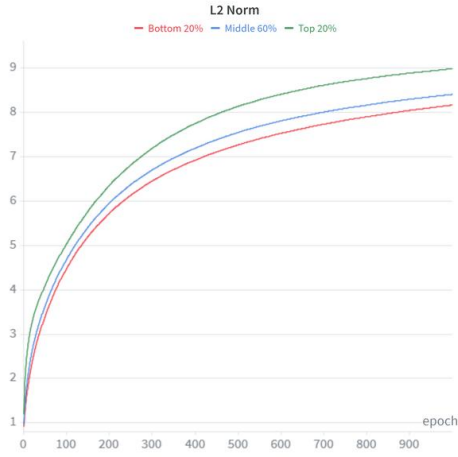
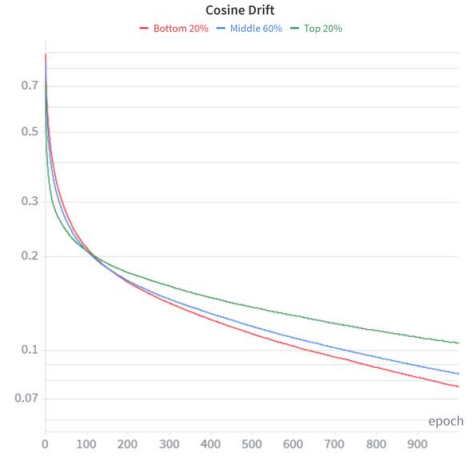
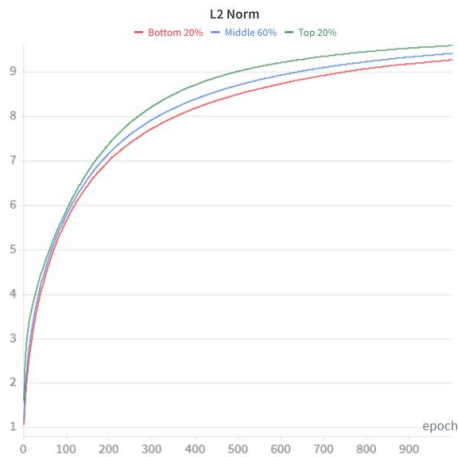
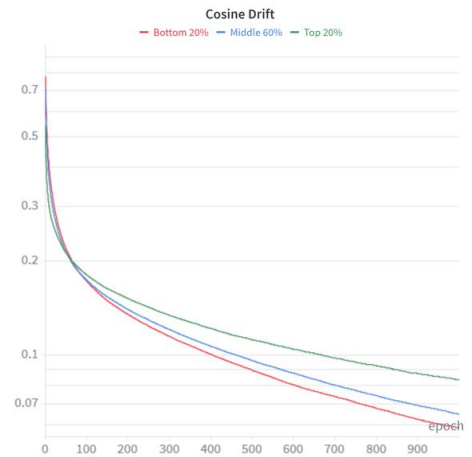
(a) L_2 Norm for $batch\ size = 2048$ (b) $Cosine\ Drift$ for $batch\ size = 2048$ (c) L_2 Norm for $batch\ size = 1024$ (d) $Cosine\ Drift$ for $batch\ size = 1024$ (e) L_2 Norm for $batch\ size = 512$ (f) $Cosine\ Drift$ for $batch\ size = 512$

Figure 6.8: Comparison of the dynamics of the L_2 norm and cosine drift ($Cosine\ Drift$) of item vectors in popularity groups for different values of the $batch\ size$ parameter in the *FastLightGCN* model.

SIS Value	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
None	0.01045	0.03744	0.28720	0.15775
0.0	0.01009	0.03412	0.28138	0.15372
0.25	0.00860	0.03057	0.27542	0.14911
0.5	0.00879	0.02702	0.26786	0.14456
0.75	0.00872	0.02328	0.25665	0.13809
1.0	0.00877	0.02162	0.24724	0.13326
NDCG@20				
None	0.00407	0.01626	0.18951	0.12836
0.0	0.00392	0.01508	0.18456	0.12461
0.25	0.00358	0.01362	0.17845	0.12057
0.5	0.00349	0.01216	0.17111	0.11536
0.75	0.00340	0.01052	0.16205	0.10901
1.0	0.00361	0.00987	0.15592	0.10478
Precision@20				
None	0.00097	0.00478	0.04976	0.04876
0.0	0.00090	0.00448	0.04910	0.04795
0.25	0.00081	0.00403	0.04831	0.04677
0.5	0.00084	0.00354	0.04707	0.04525
0.75	0.00084	0.00308	0.04521	0.04319
1.0	0.00086	0.00288	0.04361	0.04164

Table 6.9: The impact of the *SIS* parameter on the evaluation metrics of the *Fast-LightGCN* model for the configuration [2048, 2048, 2048, 2048] with a *batch size* of 2048.

clear recovery of overall metric quality and stabilization of the optimization process.

The selective targeting of hits is directly reflected in the geometry of the embedding space, which we present in Figure 6.9, illustrating the behavior of the L_2 Norm and *Cosine Drift* metrics. In the absence of smoothing ($\alpha = 1.0$), the vectors of all groups exhibit elevated, chaotic directional wandering – *Cosine Drift* reaches a value of 0.140 for the Top 20% and 0.108 for the Bottom 20%. Additionally, the L_2 norm in the Top 20% group is visibly suppressed to 7.84, and for the Bottom 20% it drops to 6.70. The application of uniform sampling ($\alpha = 0.0$) unleashes the representation potential: it allows the tail vectors to grow more strongly (their L_2 norm increases to 7.19) and stabilizes the trajectory in all studied deciles (a decrease in *Cosine Drift* to 0.129 for hits and 0.100 for the niche).

From the perspective of propagation mechanics, stochastic sampling with the threshold of $\alpha = 1.0$ unnaturally floods the sampled subgraphs with popular items. The most popular items appear in the neighborhood of almost every node, and their vectors are averaged with too many unrelated items, which blurs their unique signal

(*oversmoothing*). In practice, this acts as a destructive mechanism that suppresses *Popularity Bias*. However, the experiment proves the key role of the smoothing mechanism. Smoothly reducing the impact of the node degree using the parameter α prevents the described oversaturation and stabilizes the optimization dynamics. In *FastLightGCN*, this constitutes a powerful tool for structural regularization, allowing for the deliberate modeling of the embedding space without exposing the model to extreme estimator variance.

6.4.4 PinSage Random Walk and Dropouts

The last group of methods analyzed are various structural dropout techniques. The first is a sampling method using short random walk paths inspired by the *PinSage* model, which dynamically changes the neighborhood of each node. From an optimization perspective, this mechanism acts as an advanced structural dropout. The second and third solutions studied are *edge dropout*, which randomly removes some edges during propagation, and *node dropout*, which eliminates entire nodes along with their incident edges. By using these mechanisms, the model does not learn to rely rigidly on specific, dominant elements. Filtering out random connections reduces structural noise and significantly weakens the bias toward the most popular items.

To objectively investigate the effectiveness of graph sampling using the *PinSage* method, the research process was divided into four independent, key parts controlling the construction and utilization of the graph (the names of the tested variables are given in parentheses):

- graph rebuild frequency (*pin_sampling*),
- neighborhood size (*pin_top_k*),
- number of paths (*pin_n_traces*),
- path length (*pin_n_hops*),

All of the above tests were performed for the base model with $L = 4$ layers.

Graph reconstruction frequency

The first investigated parameter is the frequency of stochastic subgraph reconstruction (*pin_sampling*), defined as the number of epochs X after which paths are re-sampled and a new adjacency matrix is generated. This mechanism determines how long the model exploits a given topology in the training process. Analyzing its impact on the *Popularity Bias* phenomenon, we tested it for values $X \in \{1, 5, 20, 100, 1000\}$ under three different variants of the maximum neighborhood size $T \in \{25, 50, 75\}$, a constant number of neighbors ($K = 12$), and path length ($H = 4$). Below we examine the models with the best results for parameter T equal to 75.

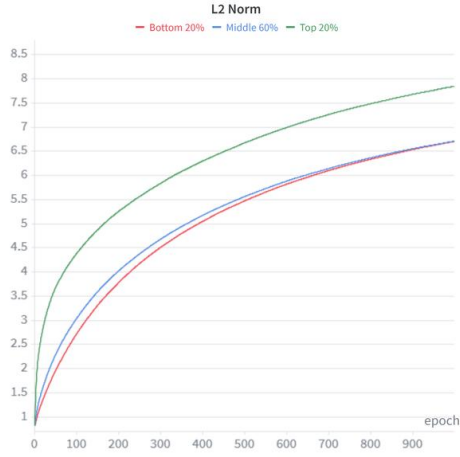
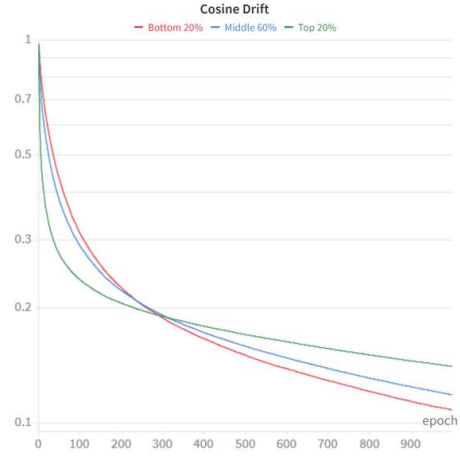
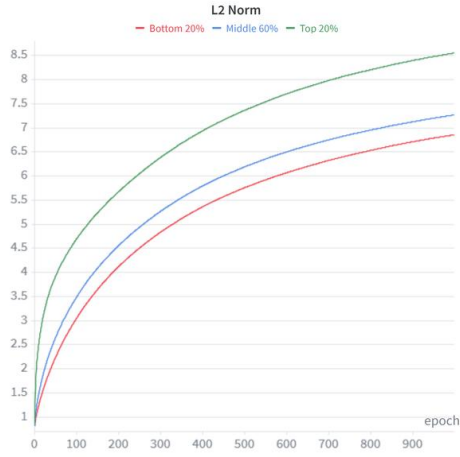
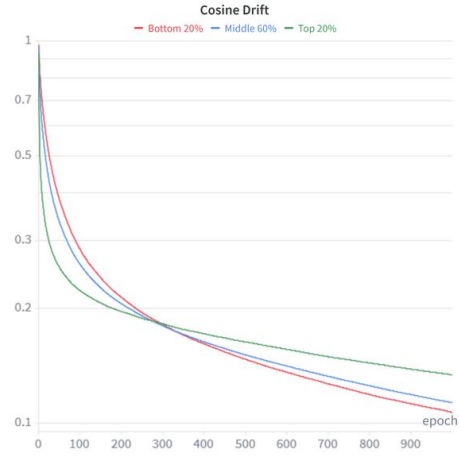
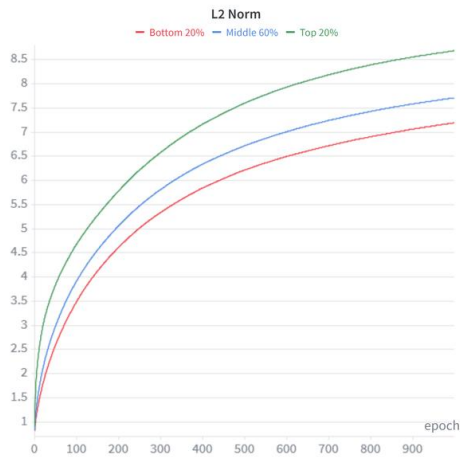
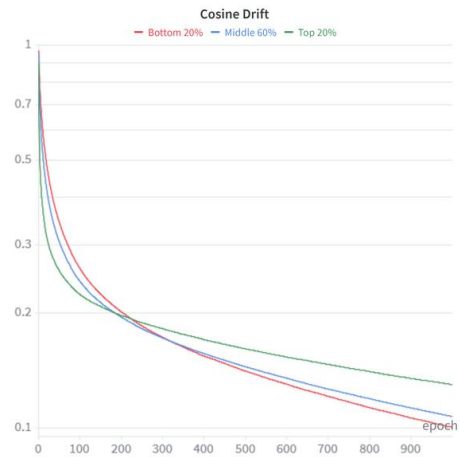
(a) L_2 Norm for $SIS = 1.0$ (b) *Cosine Drift* for $SIS = 1.0$ (c) L_2 Norm for $SIS = 0.5$ (d) *Cosine Drift* for $SIS = 0.5$ (e) L_2 Norm for $SIS = 0.0$ (f) *Cosine Drift* for $SIS = 0.0$

Figure 6.9: Comparison of the dynamics of the L_2 norm and the cosine drift (*Cosine Drift*) of item vectors across popularity groups for selected values of the SIS parameter in the *FastLightGCN* model.

Parameter X	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
1000	0.01758	0.05701	0.31758	0.18164
100	0.01662	0.05675	0.31982	0.18213
20	0.01762	0.05656	0.31632	0.18222
5	0.01778	0.05637	0.31526	0.18082
1	0.01695	0.05724	0.31602	0.18121
Baseline	0.00764	0.04903	0.33142	0.18194
NDCG@20				
1000	0.00609	0.02316	0.22420	0.15249
100	0.00566	0.02328	0.22552	0.15308
20	0.00608	0.02321	0.22410	0.15266
5	0.00611	0.02304	0.22303	0.15175
1	0.00580	0.02343	0.22225	0.15181
Baseline	0.00255	0.01916	0.23263	0.15381
Precision@20				
1000	0.00147	0.00721	0.05446	0.05537
100	0.00142	0.00723	0.05474	0.05555
20	0.00150	0.00720	0.05429	0.05536
5	0.00149	0.00709	0.05408	0.05522
1	0.00145	0.00726	0.05387	0.05506
Baseline	0.00068	0.00619	0.05655	0.05583

Table 6.10: Impact of parameter X on the evaluation metrics of the baseline model for parameters $T = 75$, $K = 12$, and $H = 4$.

Data analysis shows that the metric results for each value are very close to each other. This demonstrates high robustness to variance, since the generated graphs largely overlap topologically, providing the optimizer with a stable and reproducible gradient flow. However, at the moment of graph reconstruction, it is visible that the longer the model learns a given topology, the greater the drop in metrics is, which is subsequently gradually recovered. Figure 8.1 illustrates this situation.

The convergence of these results suggests that the choice of parameter X is primarily dictated by computational cost. Applying this mechanism in every epoch ($X = 1$), following classic dropout, with its huge overhead in time and energy, does not yield a rational gain in prediction accuracy. Refreshing the matrix every 20 epochs represents an optimal compromise point that successfully mitigates the *Popularity Bias* phenomenon, while simultaneously offering the best *Precision* value for the niche among the tested parameters.

In the following parts of the analysis, similarly to Section 6.4.3, we limit ourselves to a single configuration, which we will modify with respect to a given parameter. This configuration is $20 - 4 - 75 - 12$, which corresponds to the layout $X - H - T - K$.

Neighborhood Size

Parameter K	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
20	0.01305	0.05271	0.32324	0.18117
16	0.01522	0.05514	0.32182	0.18177
12	0.01762	0.05656	0.31632	0.18222
10	0.01921	0.05780	0.31548	0.18048
8	0.02022	0.05745	0.31074	0.17823
6	0.01784	0.05628	0.30281	0.17460
4	0.01620	0.05458	0.28885	0.16750
Baseline	0.00764	0.04903	0.33142	0.18194
NDCG@20				
20	0.00442	0.02127	0.22574	0.15159
16	0.00522	0.02236	0.22583	0.15260
12	0.00608	0.02321	0.22410	0.15266
10	0.00681	0.02352	0.22322	0.15185
8	0.00700	0.02325	0.22015	0.15010
6	0.00630	0.02351	0.21482	0.14681
4	0.00576	0.02296	0.20387	0.14028
Baseline	0.00255	0.01916	0.23263	0.15381
Precision@20				
20	0.00111	0.00669	0.05507	0.05528
16	0.00129	0.00701	0.05488	0.05545
12	0.00150	0.00720	0.05429	0.05536
10	0.00162	0.00737	0.05390	0.05492
8	0.00169	0.00724	0.05306	0.05408
6	0.00152	0.00709	0.05180	0.05286
4	0.00143	0.00679	0.04919	0.05029
Baseline	0.00068	0.00619	0.05655	0.05583

Table 6.11: Impact of the K parameter on the evaluation metrics of the baseline model for parameters $X = 20$, $H = 4$, and $T = 75$.

The second aspect investigated is the influence of node neighborhood reduction, defined by the parameter pin_top_k (or K for short). After counting the visitation frequency of individual vertices during the random walk, this mechanism retains only the K most frequently visited nodes, discarding the remaining edges. This acts restrictively, forcing the model to compress structural information. Experiments were conducted for values of $K \in \{20, 16, 12, 10, 8, 6, 4\}$.

Analysis of the collected results (Table 6.11) confirms that the investigated parameter is a key regularization mechanism. With a wide neighborhood ($K = 20$), the generated subgraph retains enough capacity to enable strong signal propagation through hits. The model very quickly returns to favoring them, while simultaneously

choking the performance gains for the niche.

The situation improves after reducing the number of neighbors to $K = 8$. This setting guarantees an optimal balance in mitigating *Popularity Bias*. Applying a targeted, structural dropout cuts off redundant, noisy edges around hubs, which directly translates to a drastic increase in prediction effectiveness in the *Bottom 20%* group compared to the baseline model, while popular nodes lose their former position.

Analysis of the L_2 Norm and *Cosine Drift* (Figure 8.3) shows that this mechanism successfully decompresses the embeddings of popular items; however, vectors from the niche group are also shortened. This happens because by reducing the number of neighbors, we restrict the flow of information, depriving the structure of valuable collaborative signals. In extreme cases (e.g., for a value of $K = 4$), this leads to a decrease in all metrics.

Number of traces

The third verified parameter is the number of traces (*pin_n_traces*, hereinafter denoted as T) initiated from each node. This parameter determines the strength of the local topology exploration, affecting the precision of the neighborhood estimation. Experiments were conducted for $T \in \{25, 50, 75, 100, 125, 150\}$.

The data show that the model’s effectiveness increases proportionally to the sample size. This is due to the fact that with too few traces (e.g., $T = 25$), the selected neighborhood is highly unstable. In such cases, counting node visits is subject to a large statistical error, causing the model to learn from noisy relations and lose the essential collaborative signal. As the parameter T is increased, the subgraph begins to stabilize. This results in an improvement in global metrics, with a tendency for the *Top 20%* group to take over the system. The *Bottom 20%* group also gains, but its effectiveness stabilizes at $T = 75$.

This phenomenon is reflected in the vector space analysis (Figure 8.4). The collected data show an inversely proportional relationship with respect to the parameter T : while model performance increases, the L_2 Norm values of all nodes decrease, and the *Cosine Drift* indicator slowly declines. This indicates that large L_2 norm values with a small random sample are artificially inflated by the statistical error of the random walk. Denser sampling filters out this noise. Further increasing the parameter ($T \geq 100$) minimizes the error to such an extent that the subgraph converges to the deterministic structure of the full graph, leading to a recurrence of the *Popularity Bias* phenomenon.

Parameter T	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
150	0.01765	0.05665	0.32083	0.18272
125	0.01727	0.05742	0.31974	0.18251
100	0.01732	0.05647	0.31839	0.18204
75	0.01762	0.05656	0.31632	0.18222
50	0.01661	0.05644	0.31591	0.18051
25	0.01568	0.05306	0.31088	0.17588
Baseline	0.00764	0.04903	0.33142	0.18194
NDCG@20				
150	0.00598	0.02314	0.22636	0.15340
125	0.00602	0.02346	0.22482	0.15312
100	0.00587	0.02287	0.22479	0.15258
75	0.00608	0.02321	0.22410	0.15266
50	0.00579	0.02320	0.22267	0.15142
25	0.00551	0.02195	0.21849	0.14755
Baseline	0.00255	0.01916	0.23263	0.15381
Precision@20				
150	0.00147	0.00718	0.05502	0.05566
125	0.00149	0.00730	0.05479	0.05562
100	0.00146	0.00717	0.05472	0.05554
75	0.00150	0.00720	0.05429	0.05536
50	0.00143	0.00724	0.05404	0.05492
25	0.00137	0.00678	0.05264	0.05330
Baseline	0.00068	0.00619	0.05655	0.05583

Table 6.12: The impact of parameter T on the evaluation metrics of the base model for parameters $X = 20$, $H = 4$, and $K = 12$.

Path Length

The last investigated parameter for graph sampling in the *PinSage* model is the length of a single path (*pin_n_hops*, i.e., H). Defining this parameter corresponds to shaping the so-called receptive field for each vertex, which is the structural radius from which the node is able to aggregate the collaborative signal during propagation. The experiments were conducted for $H \in \{1, 2, 4, 6\}$.

The evaluation analysis highlights a key trade-off dilemma between structural underrepresentation and the phenomenon of oversmoothing. Too short of a path ($H = 1$) traps niche nodes in their immediate neighborhood, which is dominated by the most popular hubs. Conversely, excessive lengthening of the path ($H = 6$) leads the structure to an information crisis. Narrowing this parameter down to the optimal value of $H = 4$ allows for obtaining the best results.

The nature of sparse graphs strongly resonates with the small-world phenomenon

Parameter H	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
6	0.01541	0.05722	0.31691	0.18071
4	0.01762	0.05656	0.31632	0.18222
2	0.01731	0.05741	0.31833	0.18135
1	0.01068	0.05899	0.31722	0.18114
Baseline	0.00764	0.04903	0.33142	0.18194
NDCG@20				
6	0.00536	0.02292	0.22252	0.15116
4	0.00608	0.02321	0.22410	0.15266
2	0.00603	0.02347	0.22569	0.15310
1	0.00353	0.02456	0.22524	0.15328
Baseline	0.00255	0.01916	0.23263	0.15381
Precision@20				
6	0.00136	0.00716	0.05422	0.05495
4	0.00150	0.00720	0.05429	0.05536
2	0.00146	0.00727	0.05465	0.05543
1	0.00089	0.00755	0.05447	0.05543
Baseline	0.00068	0.00619	0.05655	0.05583

Table 6.13: The impact of the H parameter on the evaluation metrics of the baseline model for parameters $X = 20$, $T = 75$, and $K = 12$.

(well-known in social network analysis) and the closely related theory of six degrees of separation initiated by *S. Milgram's* experiment [9].

This means that with a path length equal to six steps, the aggregation algorithm has a receptive field of almost global scope, connecting the source node in just a few hops with virtually any other object in the graph. As a result, the unique features of the profiles vanish due to the *oversmoothing* phenomenon, and the blurred representations degrade the predictive capabilities of the system.

The mechanics of this trade-off between *Popularity Bias* and *oversmoothing* is illustrated in Figure 8.5, which presents the dynamics of the latent space for the L_2 Norm metric. As the parameter H increases, the length of the embeddings for the *Top 20%* group systematically increases. This is due to the fact that hits in the expanded receptive field act as global attractors – they increment their norm by assimilating even the smallest signal, since from an optimization perspective, absorbing a mass of incompatible, common edges works to their advantage.

The *Bottom 20%* group, which is based on sparse, unique relationships, exhibits a different dynamic. If, as a result of deep exploration ($H = 6$), items from this group become linked with distant, incompatible neighbors, their hermetic signal is drowned out. As a consequence, they lose significance in the optimization function, and their

representations are shortened in confrontation with the dominant nodes.

Other Dropouts

Dropout rate	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
0 (Baseline)	0.00764	0.04903	0.33142	0.18194
0.4	0.01420	0.05593	0.32573	0.18447
0.5	0.01776	0.05830	0.32332	0.18475
0.6	0.01940	0.06188	0.31957	0.18439
0.7	0.02198	0.06396	0.31441	0.18302
0.8	0.02128	0.06415	0.30854	0.18011
0.9	0.01341	0.05208	0.30397	0.17165
NDCG@20				
0 (Baseline)	0.00255	0.01916	0.23263	0.15381
0.4	0.00488	0.02309	0.23100	0.15577
0.5	0.00607	0.02433	0.23006	0.15611
0.6	0.00690	0.02609	0.22787	0.15605
0.7	0.00791	0.02741	0.22513	0.15520
0.8	0.00786	0.02806	0.21855	0.15200
0.9	0.00505	0.02227	0.20804	0.14277
Precision@20				
0 (Baseline)	0.00068	0.00619	0.05655	0.05583
0.4	0.00122	0.00718	0.05619	0.05670
0.5	0.00150	0.00746	0.05584	0.05678
0.6	0.00164	0.00786	0.05522	0.05665
0.7	0.00186	0.00811	0.05459	0.05626
0.8	0.00178	0.00805	0.05375	0.05552
0.9	0.00113	0.00648	0.05278	0.05291

Table 6.14: Impact of *edge dropout* strength on the evaluation metrics of the baseline model.

For a reliable evaluation of advanced sampling mechanisms (*PinSage*), it is necessary to compare their performance with traditional, less complex methods of structural reduction. First, the impact of random edge dropping (*Edge Dropout*) was investigated, which we tested for six different values limiting the graph topology: $p \in \{0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$.

Analysis of the data in Table 6.14 reveals a strong impact of this method. Overall prediction metrics reach their maximum for a trade-off value ($p = 0.5$). From the perspective of the *Popularity Bias* phenomenon, the method exhibits unexpectedly high effectiveness. Thinning the adjacency matrix at the level of $p = 0.7$ allows for a clear improvement in ranking for the *Bottom 20%* group, whose effectiveness is

Dropout rate	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
0 (Baseline)	0.00764	0.04903	0.33142	0.18194
0.02	0.00830	0.04876	0.32829	0.18057
0.04	0.00942	0.05024	0.32288	0.17880
0.06	0.01098	0.05197	0.31768	0.17732
0.08	0.01167	0.05151	0.31275	0.17592
0.1	0.01321	0.05229	0.30921	0.17456
NDCG@20				
0 (Baseline)	0.00255	0.01916	0.23263	0.15381
0.02	0.00294	0.01988	0.22717	0.15132
0.04	0.00339	0.02089	0.21972	0.14778
0.06	0.00404	0.02212	0.21247	0.14451
0.08	0.00457	0.02205	0.20617	0.14155
0.1	0.00523	0.02252	0.20112	0.13892
Precision@20				
0 (Baseline)	0.00068	0.00619	0.05655	0.05583
0.02	0.00073	0.00621	0.05610	0.05551
0.04	0.00086	0.00634	0.05531	0.05492
0.06	0.00098	0.00651	0.05455	0.05438
0.08	0.00106	0.00647	0.05378	0.05381
0.1	0.00117	0.00659	0.05325	0.05340

Table 6.15: Impact of *node dropout* strength on the evaluation metrics of the baseline model.

nearly three times that of the baseline model. However, after exceeding the tolerance threshold ($p \geq 0.8$), the model experiences a decline in generalization ability, which is manifested by a collapse of predictions in the long tail once again.

This phenomenon is reflected in the analysis of the vector space (Figure 8.6). The collected data show a direct relationship between the degradation of model predictions and the behavior of embeddings for an increasing parameter p : the L_2 Norm values of all nodes expand, and the *Cosine Drift* index systematically increases. This indicates that the increase in vector lengths under almost complete edge loss is artificially forced by the optimizer to compensate for critical deficits in the collaborative signal. Elevated cosine drift values prove a loss of stability in the system. Further increasing the dropout rate ($p \geq 0.8$) stiffens the isolated vectors in undesirable regions of the space, which ultimately deprives the system of coherent relations and the previously mentioned generalization ability.

An identical dynamic was observed during the concurrent analysis of node dropping (*Node Dropout*). We conducted experiments for the parameter p equal to $\{0.02, 0.04, 0.06, 0.08, 0.1\}$. Its values are significantly smaller than in the previ-

ous case because, as Table 6.15 clearly shows, dropping entire nodes is a far more destructive operation for the structure of a sparse graph than deleting individual edges. Although this method effectively dampens the influence of hubs, nearly doubling the prediction performance of the *Bottom 20%* group, it comes at the cost of an immediate degradation of the global metrics of the entire system.

This phenomenon is confirmed by the latent space analysis (Figure 8.7). Similar to the *Edge Dropout* variant, aggressive thinning of the topology forces an increase in L_2 norms to compensate for the lost signal, and also induces embedding destabilization measured by a high drift value.

Ultimately, this demonstrates the fundamental trade-offs between the studied methods. Classical stochasticity successfully decompresses the embeddings of popular items, but blind randomness simultaneously suppresses the valuable collaborative signal in the process. On the other hand, balanced random walks (*PinSage*) precisely protect niche relationships from the *Gradient Starvation* phenomenon, but from an engineering standpoint, this approach proves to be a structurally and computationally highly complex mechanism.

6.4.5 Combined Approaches

We conclude our study by experimenting with mixtures of mechanisms to verify whether their combination yields a synergy effect or rather the opposite trend. The first proposal is the fusion of *Smoothed BPR* and *IPS* for the baseline model. The next three are combinations of the *FastLightGCN* (**FLGCN**) model with the aforementioned mechanisms:

1. *FLGCN* + *IPS*,
2. *FLGCN* + *SBPR*,
3. *FLGCN* + *SBPR* + *IPS*.

As the baseline model for *FLGCN*, the configuration [2048] and *batch size* = 2048 was chosen, as it offered the best trade-off between recommendation quality and training time.

LightGCN* + *IPS* + *SBPR

Analyzing the first of the proposed hybrids, i.e., the combination of the *IPS* and *SBPR* mechanisms applied to the classic *LightGCN* model, we observed an extremely strong synergy effect, which allows for radical control of the *Popularity Bias* phenomenon. The comparison of evaluation results indicates that the aggressive configuration of both methods ($\alpha = 1.0$, $C = 20.0$) results in an unprecedented transfer of

C	α	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20					
None	None	0.00764	0.04903	0.33142	0.18194
20.0	1.0	0.04968	0.09637	0.14010	0.11724
20.0	0.5	0.03568	0.08988	0.20153	0.13770
20.0	0.25	0.03069	0.08285	0.23244	0.15102
5.0	0.25	0.02438	0.07786	0.27318	0.16730
NDCG@20					
None	None	0.00255	0.01916	0.23263	0.15381
20.0	1.0	0.02014	0.04839	0.08573	0.09284
20.0	0.5	0.01341	0.04308	0.13367	0.10617
20.0	0.25	0.01123	0.03860	0.15926	0.12057
5.0	0.25	0.00845	0.03428	0.19344	0.13906
Precision@20					
None	None	0.00068	0.00619	0.05655	0.05583
20.0	1.0	0.00421	0.01271	0.02137	0.03480
20.0	0.5	0.00306	0.01154	0.03292	0.04043
20.0	0.25	0.00257	0.01054	0.03886	0.04510
5.0	0.25	0.00203	0.00991	0.04646	0.05076

Table 6.16: The impact of the combined parameters C and α on the evaluation metrics of the *LightGCN* model.

model effectiveness. Although the global accuracy metrics deteriorate visibly compared to the baseline variant, this masks a complete shift of the network’s attention to rarer items. The results in the Top 20% group collapse drastically, but this happens at the cost of a massive, multi-fold increase in effectiveness for the Bottom 20% group and a strong boost in accuracy for the deciles in the middle of the distribution (Middle 60%). Relaxing the hybrid hyperparameters ($\alpha = 0.25, C = 5.0$), in turn, allows for a significant recovery of the global model quality, while maintaining a very clear improvement for long-tail items.

This extreme shift of focus is directly and clearly reflected in the dynamics of the vector embedding space, presented in Figure 8.8. Observing the behavior of the milder configuration ($\alpha = 0.25, C = 20.0$) and the systematic increase of these parameters, one can notice a clear separation of the most popular items from mid-ranking and niche elements. From the perspective of the embedding space geometry, this phenomenon can be interpreted as a superposition—that is, the physical overlapping and summation—of two directed forces. Both methods modify gradients independently to weaken hits, and their fusion results in combining these vectors, hitting the most frequently chosen items with multiplied force. Consequently, the combined mechanisms jointly and extremely aggressively push popular objects out of their dominant regions of space, making room for rarer long-tail items. This clear

vector trend fully explains the observed redistribution of effectiveness, rendering any further, detailed analysis of this phenomenon redundant.

FLGCN + *IPS*

Parameter C	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20				
None	0.00446	0.02614	0.28896	0.15172
100.0	0.00675	0.02936	0.27360	0.14920
200.0	0.00783	0.02953	0.27231	0.14906
400.0	0.00776	0.02963	0.27246	0.14902
NDCG@20				
None	0.00163	0.01067	0.18935	0.12345
100.0	0.00273	0.01260	0.17224	0.12090
200.0	0.00324	0.01261	0.17173	0.12068
400.0	0.00323	0.01256	0.17201	0.12062
Precision@20				
None	0.00043	0.00330	0.04704	0.04511
100.0	0.00065	0.00365	0.04402	0.04492
200.0	0.00075	0.00367	0.04387	0.04487
400.0	0.00075	0.00368	0.04389	0.04484

Table 6.17: The impact of the combined methods *FLGCN* ([2048] with *batch size* = 2048) and *IPS* (parameter C) on the evaluation metrics.

The first of the investigated combinations of *FastLightGCN* is its fusion with the *IPS* mechanism. A comparison of the evaluation results reveals a very favorable and purposeful shift in the accuracy balance. Although at the optimal configuration (e.g., for $C = 200.0$) we observe a marginal decrease in global accuracy compared to the pure stochastic model, it directly results from the targeted weakening of the most popular items. In return, the model significantly improves the representation of the long tail, almost doubling the accuracy, and also noticeably raises the results for objects from the middle of the distribution (Middle 60%). However, it should be noted that systematically increasing the strictness of the *IPS* weight parameter (tested sequentially for $C \geq 200.0$) relatively quickly stops bringing proportional benefits. Extreme values of the parameter exhibit a halt in the improvement of results, indicating that the saturation point for the potential of this method is quickly reached.

From the perspective of embedding space geometry, presented in Figure 8.9, the behavior of this hybrid is very close to the phenomena observed earlier in the analysis of *IPS* for the classic *LightGCN* model. The reversal of roles in the charts

relative to the *baseline* (for the value $C = 1$) is clear, but this phenomenon is limited solely to the L_2 *Norm* metric. In accordance with our previous conclusions, this mechanism forcibly increases the niche vectors – their length dynamically increases, almost completely erasing the previous gap and aligning with the hits (reaching a common level in the range of 8.56 – 8.85 for $C = 200.0$).

On the other hand, in the case of the *Cosine Drift* metric, we notice the same phenomenon of a clear reversal of the drift positions of hits and niches that we diagnosed in the *IPS* analysis for the *baseline*. In pure *FastLightGCN*, the tail objects drifted the most, whereas after the fusion with *IPS*, these groups completely swap places in the chart. However, the level of cosine drift does not decrease at all, in contrast to the fusion with the classic model. This is due to the fact that in the stochastic architecture, we only update the vertices from the first layer of the sampled graph. Niche items naturally have very few edges in it, making their presence in the sampled neighborhood rare and highly unpredictable, which directly sustains a high level of their drift.

FLGCN + SBPR

Parameter α	Bottom 20%	Middle 60%	Top 20%	Overall
Recall@20				
None	0.00446	0.02614	0.28896	0.15172
0.25	0.00517	0.02724	0.28981	0.15301
0.5	0.00545	0.02935	0.29150	0.15415
0.75	0.00632	0.02986	0.28948	0.15415
1.0	0.00649	0.03006	0.28758	0.15315
1.25	0.00660	0.02996	0.28116	0.15054
NDCG@20				
None	0.00163	0.01067	0.18935	0.12345
0.25	0.00180	0.01113	0.19099	0.12516
0.5	0.00199	0.01185	0.19257	0.12632
0.75	0.00233	0.01235	0.19109	0.12598
1.0	0.00238	0.01249	0.18893	0.12410
1.25	0.00245	0.01256	0.18350	0.12146
Precision@20				
None	0.00043	0.00330	0.04704	0.04511
0.25	0.00047	0.00347	0.04780	0.04604
0.5	0.00052	0.00370	0.04832	0.04651
0.75	0.00060	0.00382	0.04816	0.04661
1.0	0.00064	0.00381	0.04772	0.04629
1.25	0.00063	0.00384	0.04664	0.04537

Table 6.18: Impact of the combined *FLGCN* ([2048] with *batch size* = 2048) and *SBPR* (parameter α) methods on evaluation metrics.

The second analyzed hybrid of *FastLightGCN* is its combination with the *Smoothed BPR* (*SBPR*) mechanism. The comparison of the evaluation results also shows a satisfactory synergistic effect in this case. While *IPS* improved the position of the niche primarily at the expense of popular items, moderate variants of *SBPR* (e.g., configurations for $\alpha \in [0.5, 1.0]$) not only noticeably increase the accuracy for rare and mid-range items, but at the same time improve the overall recommendation quality. For instance, the variant with $\alpha = 0.75$ noticeably outperforms the pure stochastic model in terms of overall accuracy, proving that smoothing the negative distribution leads to much more balanced learning of the space. Only the most aggressive of the studied values ($\alpha = 1.25$) forces greater attenuation of hits in order to maximize niche results.

The plots (Figure 8.10) show what drives the success of this method. The L_2 *Norm* metric illustrates how this mechanism manages vector magnitude. At a low parameter value ($\alpha = 0.25$), the natural hierarchy is preserved, where hits have the longest embeddings. Increasing this parameter to $\alpha = 1.0$ results in equalizing the

vector lengths – representations for all groups stabilize at a common level. In turn, the extreme setting ($\alpha = 1.25$) leads to a physical reversal of roles, where long-tail vectors become longer than the embeddings of hits.

On the *Cosine Drift* plots, we see that the *SBPR* mechanism in the stochastic environment behaves very similarly to the baseline *LightGCN* model. With the increase of the parameter α , the mechanism stably and predictably attenuates the drift of hits relative to other groups.

FLGCN + *SBPR* + *IPS*

C	α	Bottom 20%	Middle 60%	Top 20%	Global
Recall@20					
None	None	0.00446	0.02614	0.28896	0.15172
100.0	0.75	0.01022	0.03591	0.27160	0.15017
200.0	0.75	0.01162	0.03650	0.26978	0.14998
400.0	0.75	0.01165	0.03641	0.26982	0.14997
200.0	1.25	0.01384	0.03685	0.25813	0.14489
NDCG@20					
None	None	0.00163	0.01067	0.18935	0.12345
100.0	0.75	0.00410	0.01529	0.17421	0.12010
200.0	0.75	0.00469	0.01560	0.17320	0.11964
400.0	0.75	0.00473	0.01546	0.17302	0.11958
200.0	1.25	0.00576	0.01585	0.16344	0.11448
Precision@20					
None	None	0.00043	0.00330	0.04704	0.04511
100.0	0.75	0.00099	0.00445	0.04491	0.04538
200.0	0.75	0.00111	0.00454	0.04468	0.04530
400.0	0.75	0.00111	0.00453	0.04469	0.04529
200.0	1.25	0.00127	0.00460	0.04295	0.04377

Table 6.19: The impact of combined *IPS* (parameter C) and *SBPR* (parameter α) methods on evaluation metrics for the *FLGCN* model ([2048] with *batch size* = 2048).

We conclude the study with the analysis of the most advanced hybrid: *FastLightGCN* with *IPS* and *SBPR* mechanisms. This experiment was conducted using the smoothing parameter optimal for this model ($\alpha = 0.75$), systematically increasing the weight clipping threshold C (from 100.0 to 400.0). The comparison of evaluation results proves that this is a highly effective configuration. The combination of both methods allows for an impressive, almost threefold increase in accuracy for rare items and a noticeable improvement in results for the Middle 60%, with only a minimal loss in the overall quality of the model. However, we observe a clear saturation effect here – switching from the parameter $C = 200.0$ to $C = 400.0$ brings almost no

further changes in the metrics, marking the optimal limit of this method’s utility. In turn, the verification of an additional, extreme variant ($\alpha = 1.25, C = 200.0$) proves that by accepting a more pronounced drop in global accuracy and a stronger weakening of hits, the model can generate a historic, more than threefold increase in the effectiveness of recommendations for the long tail.

The plots (Figure 8.11) clearly show the source of this combination’s success. In the case of the L_2 Norm metric, these mechanisms generate a strong, progressive stratification of groups, creating a completely new, reversed hierarchy. On the one hand, *SBPR* effectively blocks the growth of vector lengths for hits, while on the other hand, *IPS* consistently lengthens niche embeddings. As a result, as the clipping threshold is increased to $C = 400.0$, rarer items (Middle 60% and Bottom 20%) systematically move away from the embeddings of the Top 20% group.

A completely different dynamic is visible in the *Cosine Drift* plots, where these phenomena can be interpreted as a model superposition of forces. By visually overlaying the plots from the two previous experiments – the *FastLightGCN* + *SBPR* variant (for $\alpha = 0.75$) and the fusion with *IPS* – it is clear that the resulting vector trajectory for the full hybrid is almost the exact sum of their effects.

6.5 Summary of Experiments

We conducted three types of experiments. The first were modifications of the *BPR* loss function at the level of changing the probability distribution for negative items and weighting positive items inversely proportional to their popularity. The second type were graph structure sampling and regularization methods: *FastGCN* adapted to the needs of sparse graphs, approximating the operation of the convolutional layer, advanced graph sampling with *PinSage* random walks, and simple *edge* and *node* dropouts. Finally, we tested hybrids of the above mechanisms. In all analyzed methods, the key was to find a compromise between improving recommendation quality for long-tail items and a controlled decrease in performance for hits and global metrics. Summarizing the above experiments, we made the following observations:

- *Smoothed BPR* and *IPS* effectively mitigate the *Popularity Bias* phenomenon. In both cases, an increase in regularization strength improves the results for long-tail items, but at the same time leads to a controlled decline in global metrics and a weakening of the dominance of the *Top 20%* group,
- in the *FastLightGCN* model, the quality of approximation of the propagation layers is of key importance:
 - deeper models are useful only when the width of the sampled layers is sufficiently large; with too narrow sampling, the approximation error increases with depth and degrades recommendation quality,

- enforcing global uniqueness of sampled nodes between layers proved to be disadvantageous, as it limits the complexity of collaborative signal propagation paths,
 - the *batch size* parameter, which is also the size of the input layer, should remain sufficiently small relative to the width of the sampled first layer to improve estimation quality and increase the frequency of gradient updates,
 - smoothing the sampling distribution (*SIS*) stabilizes training compared to sampling heavily dependent on node degree, but too aggressive smoothing does not lead to improved metrics,
- graph sampling inspired by the *PinSage* model improved results in niche and intermediate groups, and with appropriate parameter selection, also allowed maintaining competitive global results; in particular:
 - graph reconstruction can be performed relatively infrequently, which reduces computational cost without significant loss of quality,
 - neighborhood size should be moderate to filter out redundant edges around popular nodes,
 - the number of random walks must be large enough to ensure a stable and low-noise neighborhood,
 - the length of the walks should not be too long, as this leads to the *over-smoothing* phenomenon,
 - *edge dropout* proved to be one of the most practical methods for graph structure regularization: it improved results for the *Bottom 20%* and *Middle 60%* groups, weakened the dominance of hits, and at moderate parameter values, also allowed obtaining very good global metrics,
 - *node dropout* performed worse than *edge dropout*, which results from its more destructive impact on the structure of the sparse graph; the improvement for less popular groups was limited and came at the cost of greater degradation of global quality,
 - hybrid approaches showed a synergy effect. Combining structural methods with modifications of the loss function proved particularly beneficial, though in the case of simultaneous use of *SBPR* and *IPS*, it was necessary to maintain moderate parameter values to avoid an excessive decline in global metrics.

Chapter 7

Summary

In this thesis, we focused on the problem of *Popularity Bias* in recommender systems based on graph neural networks. We assumed that the data structure and the model training process lead to favoring popular items at the expense of items belonging to the long tail. The objective of the thesis was to investigate whether it is possible to mitigate this phenomenon by modifying the loss function and the graph structure.

In this work, we described the *LightGCN* architecture and used it as a base model for the conducted experiments. We tested methods belonging to the *in-processing* category, including *Smoothed BPR*, *Inverse Propensity Scoring*, our proposed *FastLightGCN* model, sampling inspired by *PinSage*, and *edge* and *node dropout* techniques. We also investigated combinations of selected methods to check whether it is possible to achieve synergy.

The conducted experiments showed that mitigating *Popularity Bias* is possible, but it requires maintaining a trade-off between improving results for niche items and maintaining overall quality. We achieved the best results with appropriately selected loss function modifications, sampling inspired by *PinSage*, *edge dropout*, and hybrid methods. At the same time, the results confirmed that this problem is complex and cannot be solved by a single universal mechanism.

In the future, it would be worthwhile to extend the research to other datasets. An interesting direction would also be a more detailed investigation of the mechanism of sampling positive and negative examples in the *BPR* function, by moving away from the standard scheme in favor of controlled selection of pairs between popularity groups, for example: niche-hit, niche-Middle, hit-hit, or Middle-hit. Such an approach could enable more precise control over the ranking relationships between items and deliberate weakening of the dominance of popular elements.

Chapter 8

Appendix

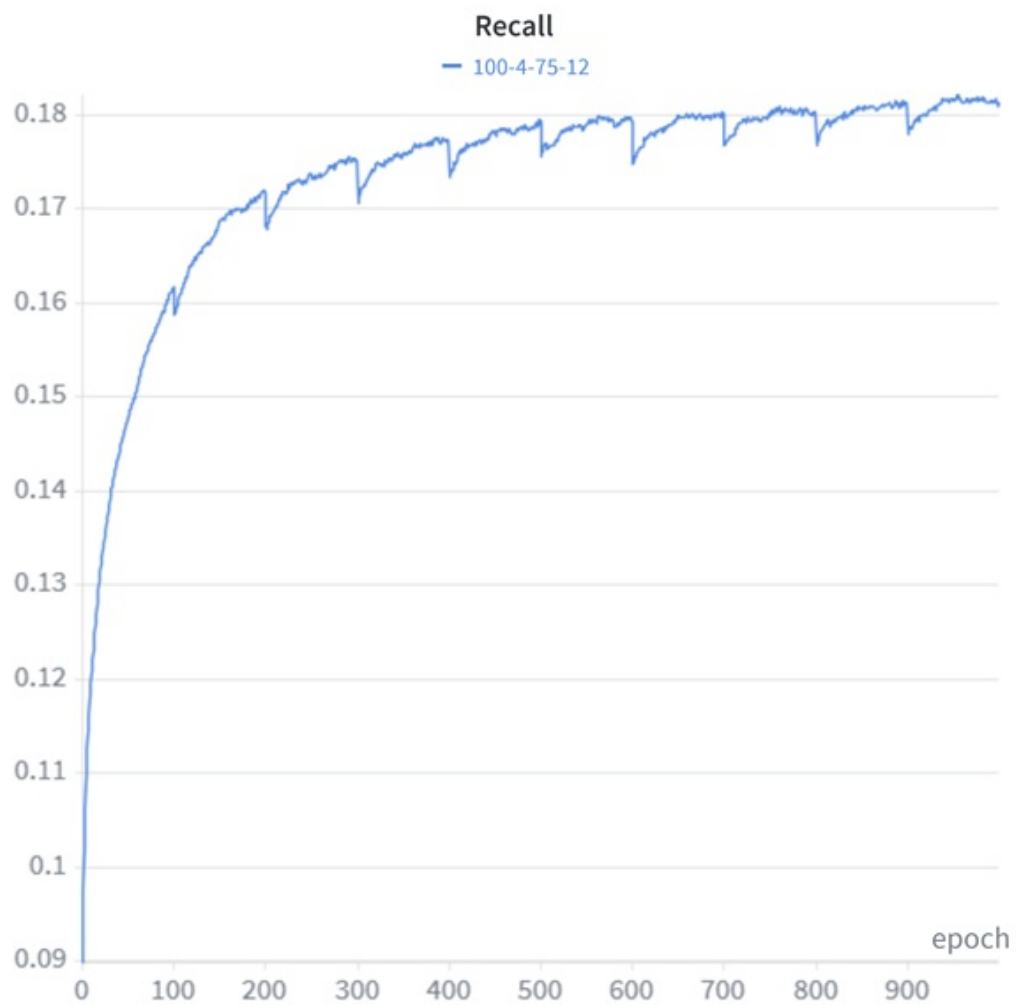
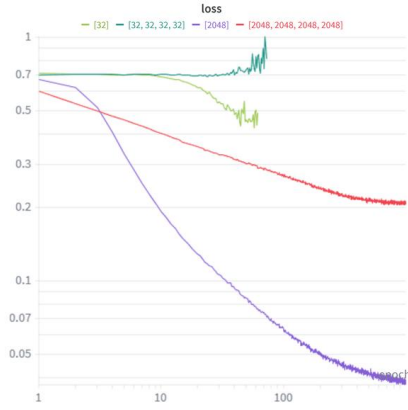
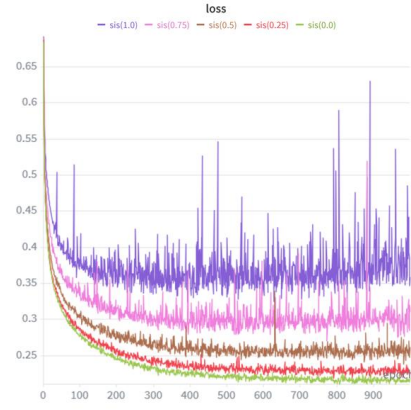


Figure 8.1: Representation of the *overfitting* phenomenon for the parameter value $X = 100$

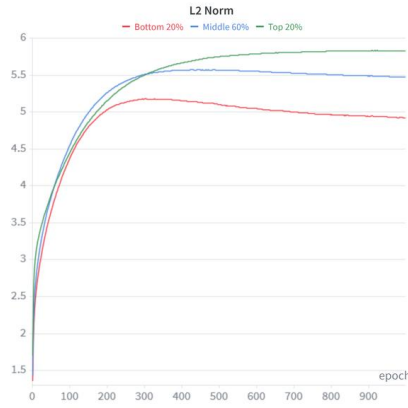


(a) Extreme depths and widths of the sampled graph

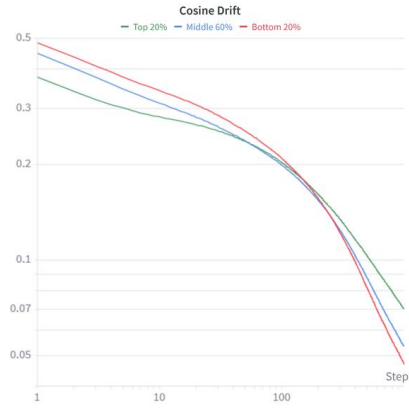


(b) Different values of the *SIS* probability distribution

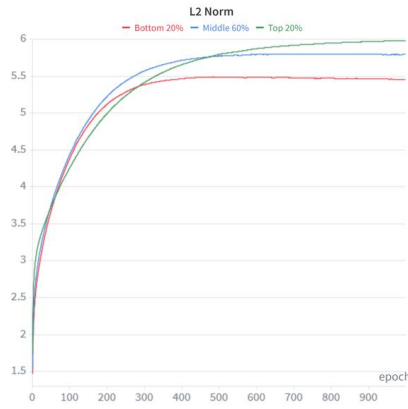
Figure 8.2: Loss function plots of the *FastLightGCN* model



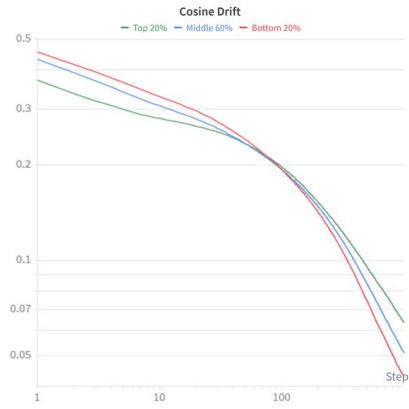
(a) L_2 Norm for $K = 8$



(b) *Cosine Drift* for $K = 8$



(c) L_2 Norm for $K = 20$



(d) *Cosine Drift* for $K = 20$

Figure 8.3: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for different values of the parameter K for *PinSage* graph sampling

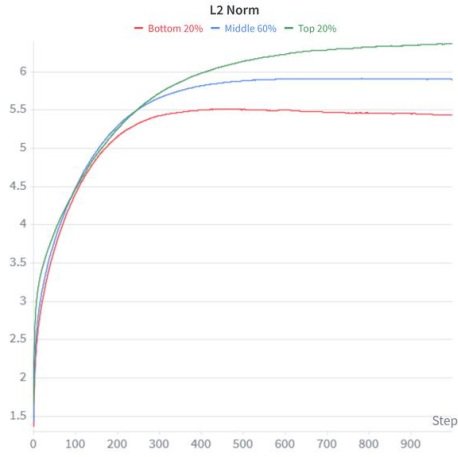
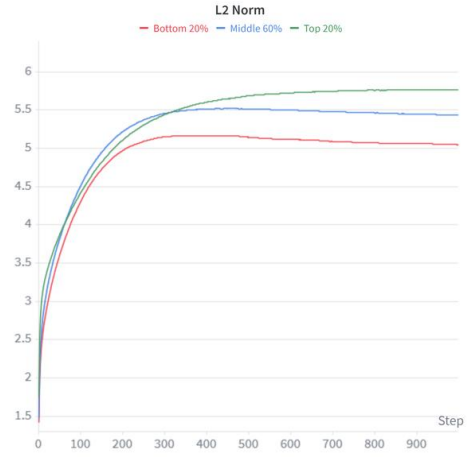
(a) L_2 Norm for $T = 25$ (b) L_2 Norm for $T = 125$

Figure 8.4: Comparison of the dynamics of the L_2 norm of item vectors in popularity groups for different values of the parameter T for *PinSage* graph sampling

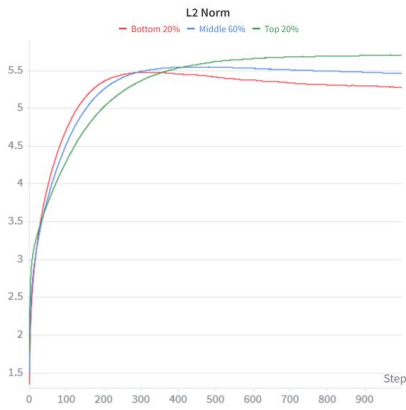
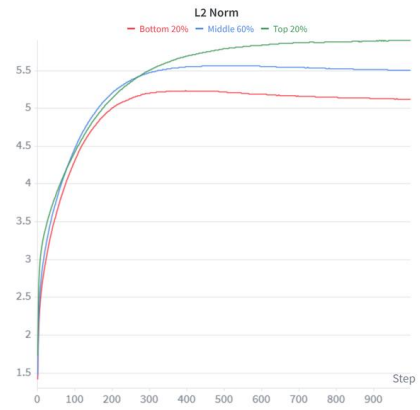
(a) L_2 Norm for $H = 1$ (b) L_2 Norm for $H = 6$

Figure 8.5: Comparison of the dynamics of the L_2 norm of item vectors in popularity groups for different values of the parameter H for *PinSage* graph sampling

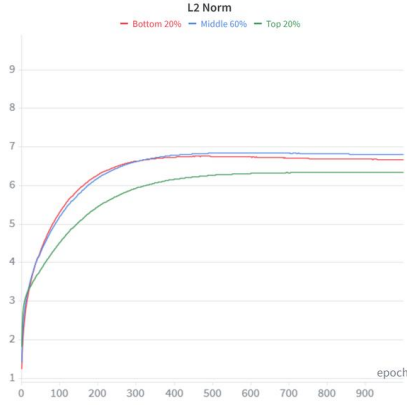
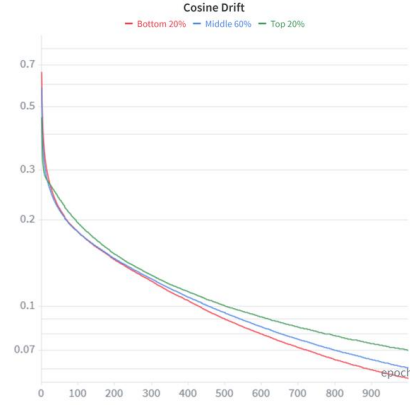
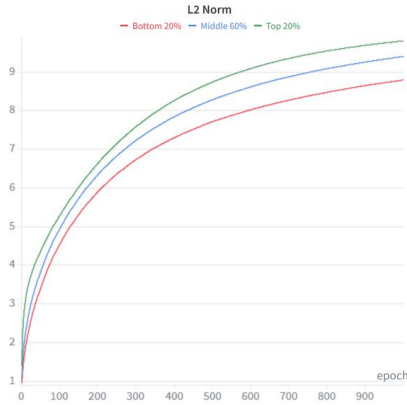
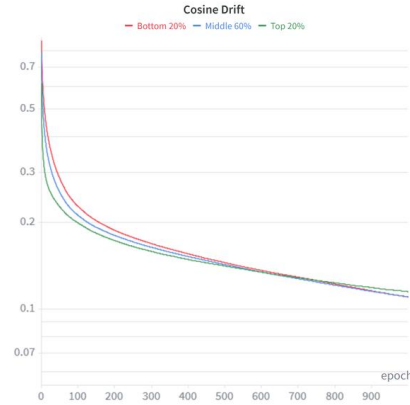
(a) L_2 Norm for $p = 0.4$ (b) *Cosine Drift* for $p = 0.4$ (c) L_2 Norm for $p = 0.9$ (d) *Cosine Drift* for $p = 0.9$

Figure 8.6: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for different values of the parameter p for *Edge Dropout*

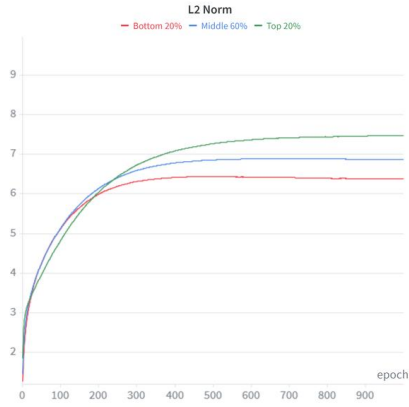
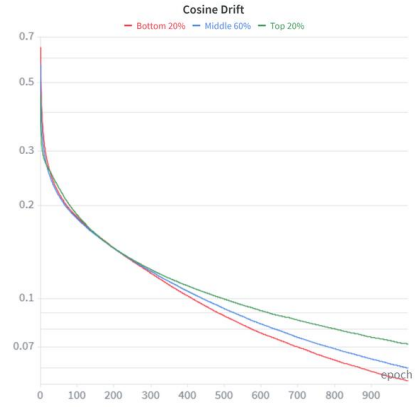
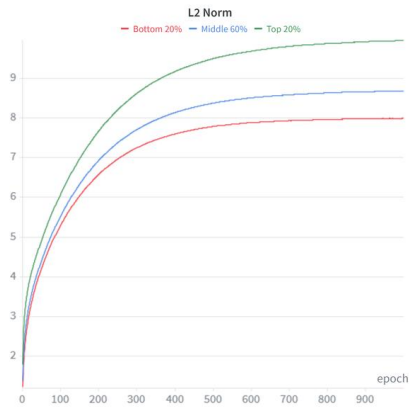
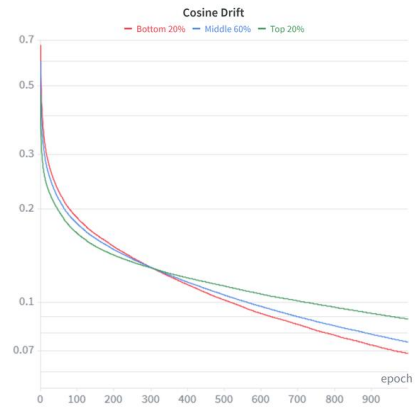
(a) L_2 Norm for $p = 0.02$ (b) *Cosine Drift* for $p = 0.02$ (c) L_2 Norm for $p = 0.1$ (d) *Cosine Drift* for $p = 0.1$

Figure 8.7: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for different values of the parameter p for *Node Dropout*

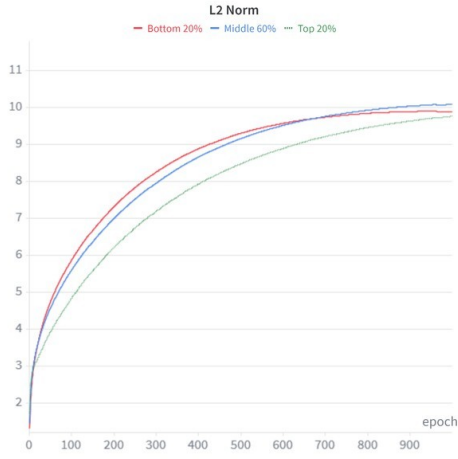
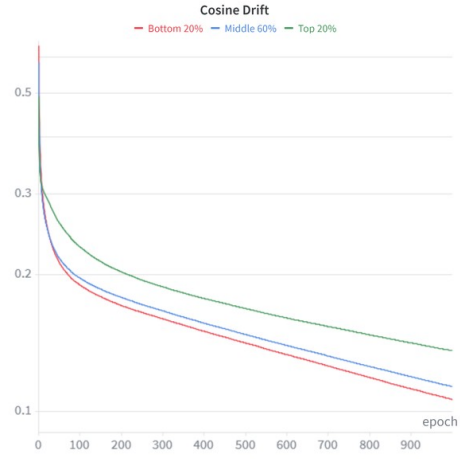
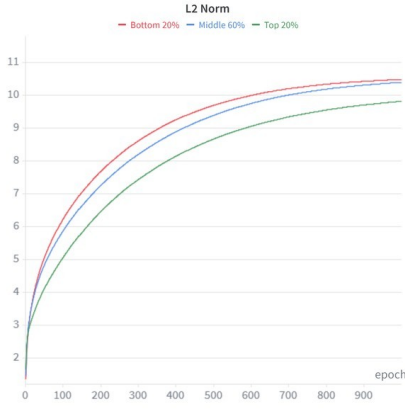
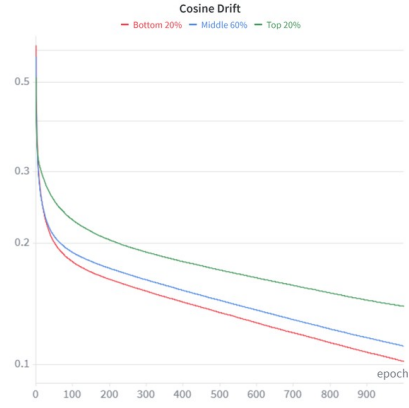
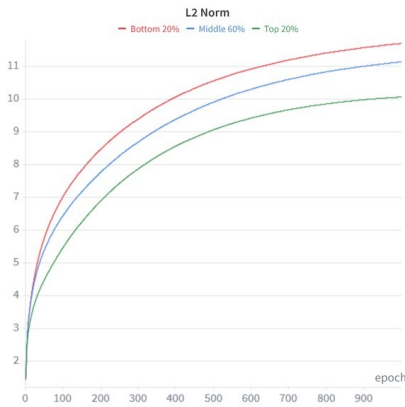
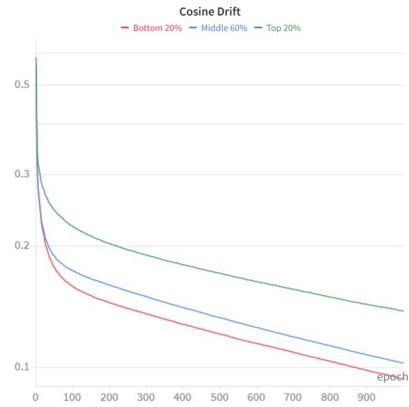
(a) L_2 Norm for $\alpha = 0.25$ and $C = 20$ (b) *Cosine Drift* for $\alpha = 0.25$ and $C = 20$ (c) L_2 Norm for $\alpha = 0.5$ and $C = 20$ (d) *Cosine Drift* for $\alpha = 0.5$ and $C = 20$ (e) L_2 Norm for $\alpha = 1.0$ and $C = 20$ (f) *Cosine Drift* for $\alpha = 1.0$ and $C = 20$

Figure 8.8: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for selected configurations of the combined *SBPR* and *IPS* methods.

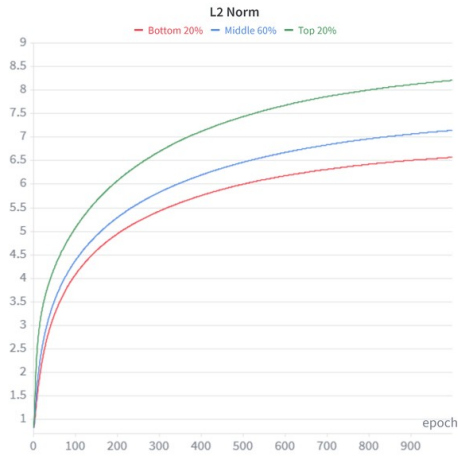
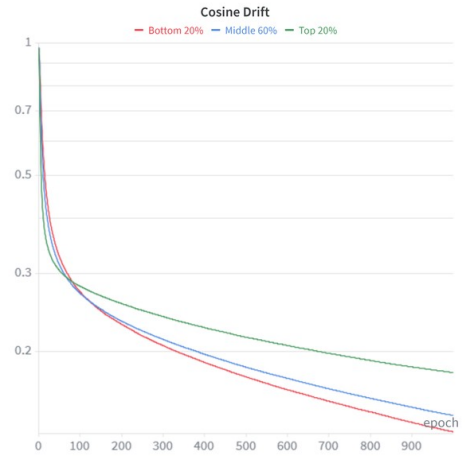
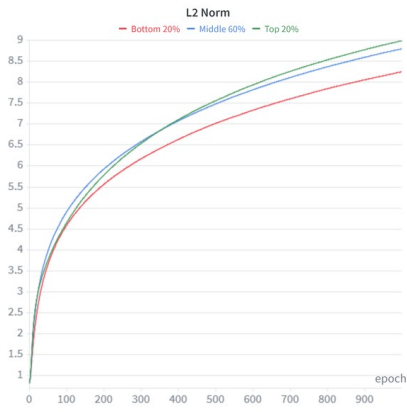
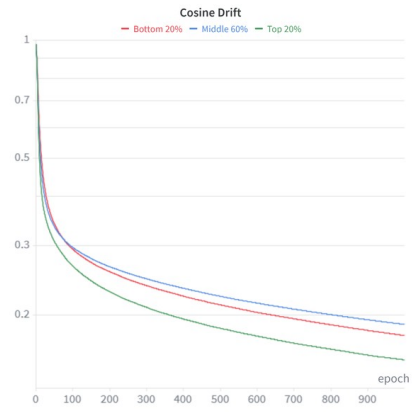
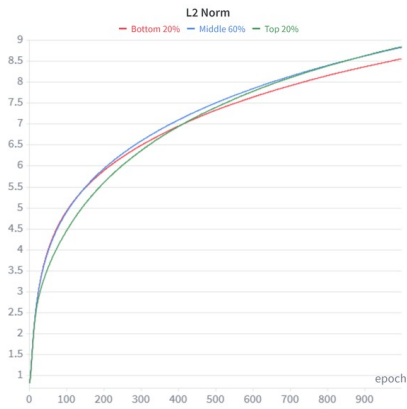
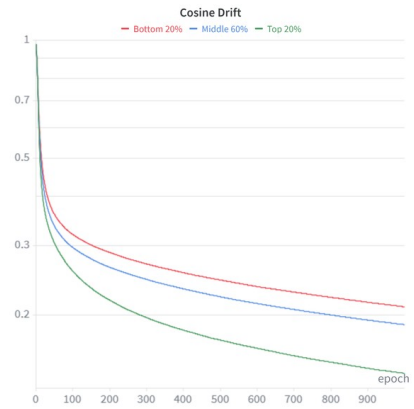
(a) L_2 Norm for the base model(b) *Cosine Drift* for the base model(c) L_2 Norm for $C = 100$ (d) *Cosine Drift* for $C = 100$ (e) L_2 Norm for $C = 200$ (f) *Cosine Drift* for $C = 200$

Figure 8.9: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for the base *FLGCN* model ([2048] with *batch size* = 2048) and variants with the *IPS* method for different values of the parameter C .

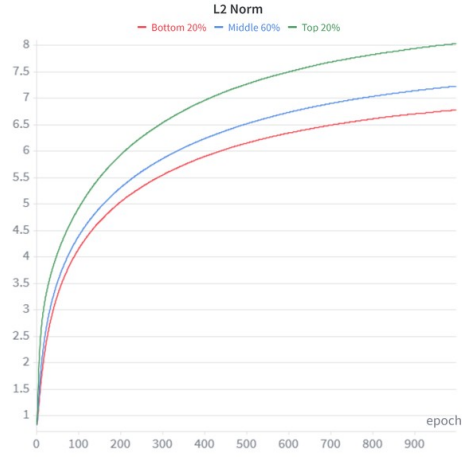
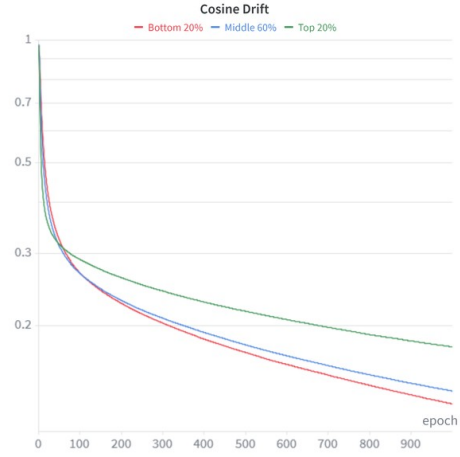
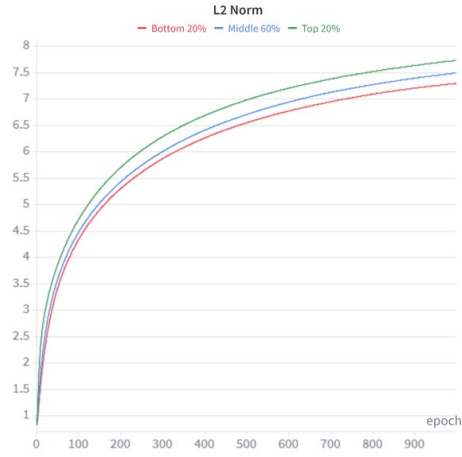
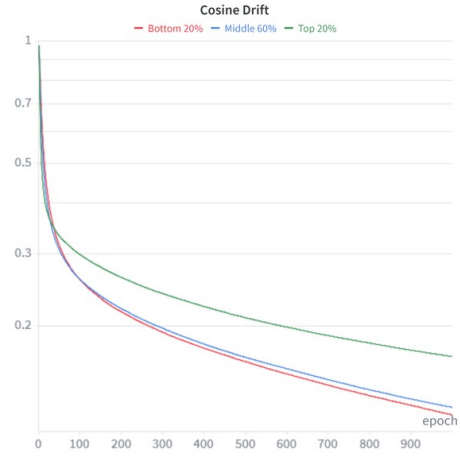
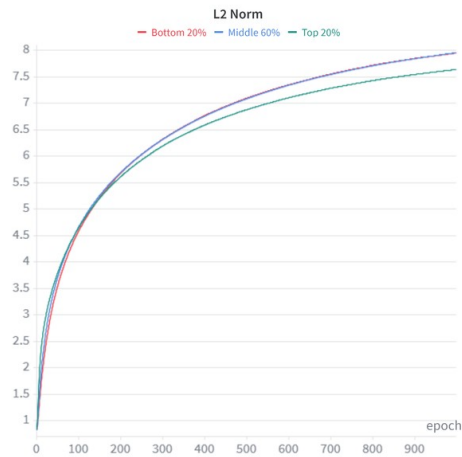
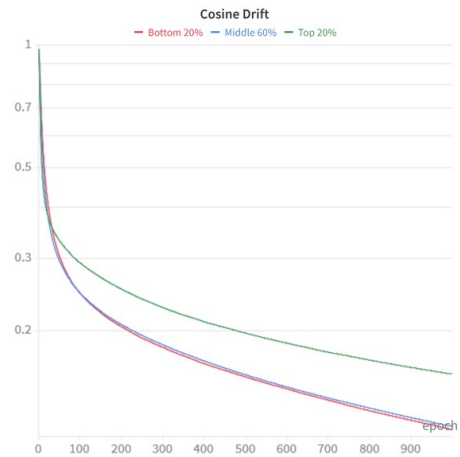
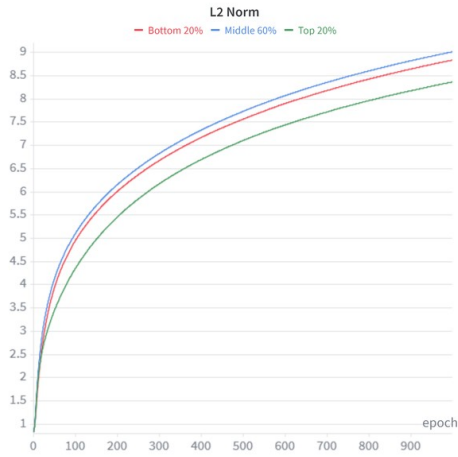
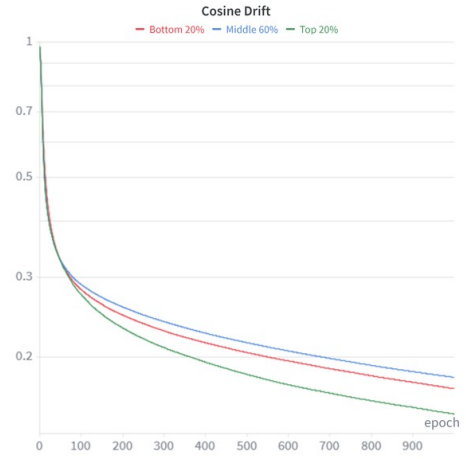
(a) L_2 Norm for $\alpha = 0.25$ (b) *Cosine Drift* for $\alpha = 0.25$ (c) L_2 Norm for $\alpha = 0.75$ (d) *Cosine Drift* for $\alpha = 0.75$ (e) L_2 Norm for $\alpha = 1.25$ (f) *Cosine Drift* for $\alpha = 1.25$

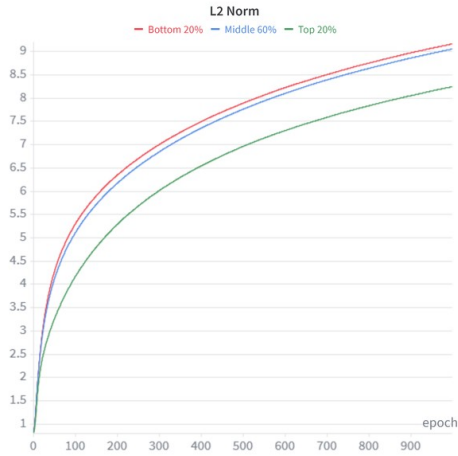
Figure 8.10: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for the hybrid $FLGCN + SBPR$ model for different values of the parameter α .



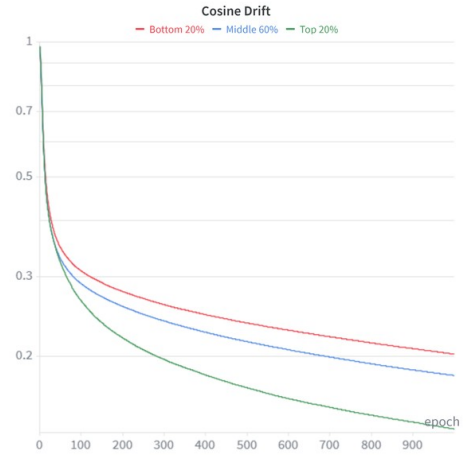
(a) L_2 Norm for $\alpha = 0.75$ and $C = 100.0$



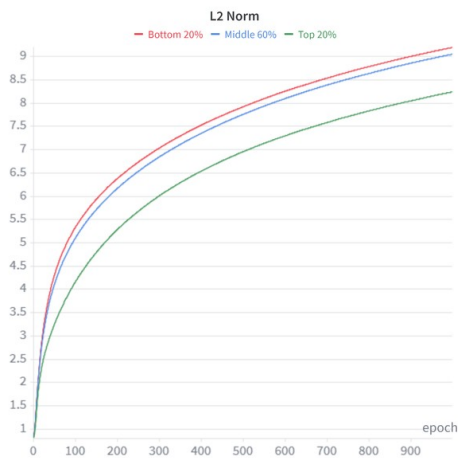
(b) *Cosine Drift* for $\alpha = 0.75$ and $C = 100.0$



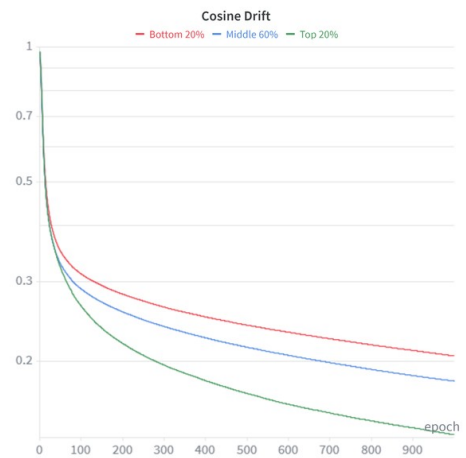
(c) L_2 Norm for $\alpha = 0.75$ and $C = 200.0$



(d) *Cosine Drift* for $\alpha = 0.75$ and $C = 200.0$



(e) L_2 Norm for $\alpha = 0.75$ and $C = 400.0$



(f) *Cosine Drift* for $\alpha = 0.75$ and $C = 400.0$

Figure 8.11: Comparison of the dynamics of the L_2 norm and cosine drift (*Cosine Drift*) of item vectors in popularity groups for the hybrid $FLGCN + IPS + SBPR$ model with a constant parameter value $\alpha = 0.75$ and different configurations of the limit parameter C .

Bibliography

- [1] Francesco Casalegno. Recommender systems: A complete guide to machine learning models. <https://medium.com/data-science/recommender-systems-a-complete-guide-to-machine-learning-models-96d3f94ea748>, 2020.
- [2] Jie Chen, Tengfei Ma, and Cao Xiao. Fastgcn: Fast learning with graph convolutional networks via importance sampling. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL: <https://openreview.net/forum?id=rytstxWAW>.
- [3] Paolo Cremonesi, Franca Garzotto, Roberto Pagano, and Massimo Quadrana. Recommending without short head. In *Companion Proceedings of the 23rd International Conference on World Wide Web*, pages 245–246. ACM, 2014. doi:10.1145/2567948.2577286.
- [4] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 639–648. ACM, 2020. doi:10.1145/3397271.3401063.
- [5] Wei Ju, Siyu Yi, Yifan Wang, Zhiping Xiao, Zhengyang Zhou, Liang Chen, Hao Wang, and Ming Zhang. A survey of graph neural networks in real world: Imbalance, noise, privacy and ood challenges. *CoRR*, abs/2403.04468, 2024. arXiv:2403.04468, doi:10.48550/arXiv.2403.04468.
- [6] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 197–206. IEEE, 2018. URL: <https://arxiv.org/abs/1808.09781>, doi:10.1109/ICDM.2018.00035.
- [7] Anastasiia Klimashevskaya, Dietmar Jannach, Mehdi Elahi, and Christoph Trattner. A survey on popularity bias in recommender systems. *arXiv preprint*

- arXiv:2308.01118*, 2023. URL: <https://arxiv.org/abs/2308.01118>, doi: 10.48550/arXiv.2308.01118.
- [8] Venkata Harshit Koneru, Xenija Neufeld, Sebastian Loth, and Andreas Grün. Enhancing recommendation quality of the sasrec model by mitigating popularity bias. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pages 780–782. ACM, 2024. doi:10.1145/3640457.3688044.
- [9] Stanley Milgram. The small world problem. *Psychology Today*, 1(1):60–67, 1967. URL: <https://snap.stanford.edu/class/cs224w-readings/milgram67smallworld.pdf>.
- [10] Francesco Ricci, Lior Rokach, and Bracha Shapira, editors. *Recommender Systems Handbook*. Springer, New York, 2022. doi:10.1007/978-1-0716-2197-4.
- [11] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural graph collaborative filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2019, Paris, France, July 21-25, 2019*, pages 165–174. ACM, 2019. doi:10.1145/3331184.3331267.
- [12] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, pages 974–983. ACM, 2018. doi:10.1145/3219819.3219890.
- [13] Ziwei Zhu, Yun He, Xing Zhao, Yin Zhang, Jianling Wang, and James Caverlee. Popularity-opportunity bias in collaborative filtering. In *Proceedings of the Fourteenth ACM International Conference on Web Search and Data Mining*, pages 85–93. ACM, 2021. doi:10.1145/3437963.3441820.
- [14] Difan Zou, Ziniu Hu, Yewen Wang, Song Jiang, Yizhou Sun, and Quanquan Gu. Layer-dependent importance sampling for training deep and large graph convolutional networks. *CoRR*, abs/1911.07323, 2019. URL: <https://arxiv.org/abs/1911.07323>, arXiv:1911.07323.